EH/Unwind Analyzer

Актуализация документа для релиза 1.0.0

Документ: EH_Unwind_Analyzer_Защита_проекта.pdf

Дата финальной проверки: 19 июня 2026 года

Этот PDF сохранен как теоретический и презентационный материал проекта.
Актуальный статус релиза подтвержден отдельными evidence-документами:

- MinGW-w64 Release CTest: 16/16 PASS
- MSVC Release CTest: 16/16 PASS
- Real benign static corpus: 106/106 valid, non-zero scores: 0
- Labeled static corpus: 14/14 valid, 12/12 anomalous samples flagged
- Dynamic smoke: 4/4 PASS
- Full Windows dynamic corpus: 124/124 valid reports, unexpected failures: 0

Актуальные release-файлы: release/RELEASE_STATUS_1.0.0_RU.md, release/VERIFIED_WINDOWS_BUILD.json и AUDIT_INVESTOR_READINESS_RU.md.

Оригинальное содержание документа начинается со следующей страницы.

Generated as a non-destructive 1.0.0 cover page; original PDF pages are preserved after this page.

EH/UNWIND ANALYZER

ПОЛНАЯ ЗАЩИТА ПРОЕКТА

От технической основы до ответов жёсткой комиссии

Теория и устройство проекта, 27 признаков и 12 внутренних аномалий, точность и ограничения модели, динамический анализ, архитектура, тестирование, аналоги, рынок, актуальность, новизна, самостоятельность реализации, сценарий выступления, демонстрация и 332 подготовленных вопроса с ответами.

Версия проекта: 1.0.0-rc1
Дата актуализации: 9 июня 2026 года
Область: PE32+ AMD64, UNWIND_INFO v1/v2; Windows-only dynamic collector

Главный принцип защиты

Не преувеличивать возможности. Сильная позиция строится на конкретных invariants, воспроизводимых результатах, честном разграничении фактов и эвристик и готовности показать, как проверяется каждое утверждение.

Содержание полной защиты

Раздел	Содержание
1–5	Постановка, термины, PE32+, RUNTIME_FUNCTION/UNWIND_INFO, parser.
6–8	27 признаков, score/confidence, динамика, reports и Ghidra.
9–12	Архитектура, тестирование, платформы, базовые провокационные вопросы.
13	Аналоги, конкуренты, SWOT, источники и 36 вопросов.
14	Актуальность, новизна, значимость и 30 вопросов.
15	Рынок, собственное ядро, open source, AI/ML и 42 враждебных вопроса.
16	Обоснование 27 findings, coverage, точность модели и 42 вопроса.
17	Слайды, текст выступления, тактика ответа и финальная памятка.
18–22	Демонстрация, fallback, карта доказательств, чек-листы и итоговая позиция.

Ключевые проверенные показатели

Показатель	Значение	Правильная интерпретация
Итоговые findings	27	Независимые аналитические classes, не все low-level checks.
Внутренние UA_* reasons	12	Детали декодирования UNWIND_INFO.
Вопросы и ответы	332	От основ до hostile defense.
Benign corpus	106/106	Наблюдаемая specificity 100% только на выбранном corpus.
Runtime functions	174 795	Разобраны без parse failures/structural anomalies на corpus.
Extended mutations	1 000	Robustness, не statistical detection accuracy.
Windows reports	124/124	Schema/semantic-valid в выполненной validation.

Как пользоваться документом

Сначала изучите разделы 1–17. Перед защитой повторите 17.3, 19, 20 и 21. На прямой вопрос отвечайте первым предложением соответствующего ответа; подробность добавляйте только после уточнения комиссии.

1. Общая постановка, актуальность и результат

В этом разделе: 12 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 1. Как называется проект?

Ответ. Краткое название — EH/Unwind Analyzer. Точная русская формулировка: инструмент структурного анализа и выявления аномалий в метаданных обработки исключений и раскрутки стека PE32+ x64.

Вопрос 2. Как одной фразой объяснить, что делает программа?

Ответ. Она разбирает служебные EH/Unwind-структуры Windows x64, проверяет их целостность, выделяет подозрительные признаки и формирует отчёт для реверс-инженера.

Вопрос 3. В чём проблема, которую решает работа?

Ответ. EH/Unwind-метаданные критичны для обработки исключений и восстановления стека, но при ручном анализе содержат много связанных адресов, флагов и переменных структур. Проект автоматизирует их безопасный разбор, формальные проверки и первичную приоритизацию подозрительных участков.

Вопрос 4. Почему тема актуальна?

Ответ. Эти структуры присутствуют в большом количестве Windows x64-программ, используются ОС во время исключений и интересны при анализе повреждённых, упакованных и нестандартно собранных файлов. Автоматизация уменьшает объём ручной работы и делает выводы воспроизводимыми.

Осторожно: Не утверждайте, что любая нестандартная unwind-структура означает вредоносность.

Вопрос 5. Кто является целевым пользователем?

Ответ. Реверс-инженеры, специалисты по анализу исполняемых файлов, исследователи защитного ПО и разработчики инструментов цифровой криминалистики. Это профессиональный аналитический инструмент, а не массовый пользовательский продукт.

Вопрос 6. Какова цель ВКР?

Ответ. Разработать программный инструмент, который безопасно анализирует PE32+ AMD64, декодирует RUNTIME_FUNCTION и UNWIND_INFO, выявляет интерпретируемые аномалии, собирает ограниченные динамические свидетельства на Windows и формирует проверяемые отчёты.

Вопрос 7. Каковы основные задачи работы?

Ответ. Изучить PE/COFF и Windows x64 exception handling; реализовать bounds-safe parser; декодировать unwind-записи; определить матрицу признаков; создать статический и динамический режимы; реализовать отчёты; подготовить тесты, корпусы и воспроизводимую поставку.

Вопрос 8. Что является объектом исследования?

Ответ. Исполняемые образы PE32+ AMD64 и содержащиеся в них метаданные обработки исключений и раскрутки стека.

Вопрос 9. Что является предметом исследования?

Ответ. Структурные инварианты RUNTIME_FUNCTION и UNWIND_INFO, признаки их повреждения или нетипичной организации, а также методы статической и динамической проверки этих признаков.

Вопрос 10. Каков практический результат?

Ответ. Рабочее C17-ядро с CLI, JSON/HTML/CSV-отчётами, Ghidra-импортом, Windows-динамикой, тестами и релизными пакетами. Оно может использоваться самостоятельно или быть подключено к backend, базе данных и frontend через JSON-контракт.

Вопрос 11. В чём новизна работы?

Ответ. Корректнее говорить о прикладной новизне: в одном воспроизводимом инструменте объединены глубокий разбор x64 unwind-метаданных, интерпретируемая матрица аномалий, ослабляющие правила, динамическая корреляция, строгая схема отчёта и интеграция с Ghidra.

Осторожно: Не заявляйте, что в мире не существует аналогов или что создан первый такой анализатор.

Вопрос 12. Чем проект отличается от обычного PE-viewer?

Ответ. PE-viewer обычно отображает структуры. Проект дополнительно проверяет взаимные инварианты, агрегирует признаки по всему образу, рассчитывает уровень подозрительности, сохраняет причины ошибок, формирует формализованный отчёт и может собирать runtime-свидетельства.

2. Базовые термины и модель исполнения

В этом разделе: 14 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 13. Что такое PE?

Ответ. Portable Executable — формат объектных и исполняемых файлов Windows. Он описывает заголовки, секции, таблицы данных и параметры, необходимые загрузчику операционной системы.

Вопрос 14. Что означает PE32+?

Ответ. PE32+ — 64-разрядный вариант PE. Знак плюс не означает расширение возможностей вообще, а отличает формат 64-разрядных образов от PE32.

Вопрос 15. Почему вы говорите AMD64 и x64?

Ответ. В контексте проекта это обозначения 64-разрядной архитектуры x86-64, для которой Windows определяет конкретный ABI и формат unwind-метаданных. Проект не переносит эти правила автоматически на ARM64 или x86.

Вопрос 16. Что такое ABI?

Ответ. Application Binary Interface определяет машинный контракт между скомпилированными компонентами: регистры, передачу аргументов, выравнивание стека, сохранение регистров, вызовы функций и правила исключений.

Вопрос 17. Что такое стек?

Ответ. Стек — область памяти потока, используемая для адресов возврата, локальных данных, сохранённых регистров и служебного состояния вызовов. На x64 он изменяется в соответствии с ABI и действиями пролога функции.

Вопрос 18. Что такое stack unwinding?

Ответ. Раскрутка стека — восстановление состояния предыдущих кадров вызовов при обработке исключения или при получении stack trace. Система должна понять, как конкретная функция изменила RSP и какие регистры сохранила.

Вопрос 19. Что означает EH?

Ответ. Exception Handling — обработка исключений. В Windows x64 она включает поиск обработчика, виртуальную раскрутку кадров и передачу управления соответствующей логике обработки.

Вопрос 20. Что такое пролог функции?

Ответ. Пролог — начальная последовательность инструкций, которая резервирует стек и сохраняет nonvolatile-регистры. Unwind-коды описывают обратные действия для поддерживаемого пролога.

Вопрос 21. Что такое эпилог?

Ответ. Эпилог восстанавливает состояние кадра перед возвратом. В unwind-модели важно корректно распознавать участки, где функция уже частично отменила действия пролога.

Вопрос 22. Что такое leaf function?

Ответ. Leaf function в данном контексте не требует полноценной записи unwind для восстановления nonvolatile-состояния. Поэтому отсутствие RUNTIME_FUNCTION не всегда является ошибкой и в проекте рассматривается как слабый контекстный признак.

Вопрос 23. Что такое nonleaf function?

Ответ. Это функция, состояние которой нельзя корректно восстановить только по адресу возврата и текущему RSP. Для неё обычно требуется описание в таблице функций и UNWIND_INFO.

Вопрос 24. Что такое volatile и nonvolatile регистры?

Ответ. Volatile-регистры вызывающая сторона не должна считать сохранёнными после вызова. Nonvolatile-регистры обязана сохранять вызываемая функция, и unwind-метаданные должны позволять восстановить их прежние значения.

Вопрос 25. Что такое first-chance и second-chance exception?

Ответ. First chance — первое уведомление отладчика об исключении до обычного поиска обработчика. Second chance возникает, если исключение не было обработано; это более сильный runtime-сигнал проблемы, но само по себе не доказывает вредоносность.

Вопрос 26. Почему обработка исключений связана с реверс-инжинирингом?

Ответ. Метаданные раскрывают диапазоны функций, прологи, сохранённые регистры, обработчики и цепочки. Они помогают восстанавливать структуру программы и обнаруживать расхождения между ожидаемым компиляторным layout и фактическим файлом.

3. Адреса и устройство PE32+

В этом разделе: 12 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 27. Чем отличаются file offset, RVA и VA?

Ответ. File offset — позиция байта в файле. RVA — смещение относительно ImageBase загруженного образа. VA — фактический виртуальный адрес в процессе: обычно ImageBase плюс RVA.

Вопрос 28. Почему нельзя читать RVA как смещение в файле?

Ответ. Секции имеют разные файловые и виртуальные диапазоны, выравнивания и размеры. Для преобразования RVA требуется найти соответствующую секцию и проверить границы, иначе parser прочитает неверные или чужие байты.

Вопрос 29. Что такое ImageBase?

Ответ. Предпочтительный базовый виртуальный адрес образа. При ASLR реальная база загрузки может отличаться, поэтому статические структуры обычно используют RVA, а динамический collector учитывает фактическую базу.

Вопрос 30. Что такое секция PE?

Ответ. Секция — диапазон файла и виртуального образа с именем, размерами, адресом и характеристиками доступа. Типичные примеры: .text, .rdata, .pdata и .xdata.

Вопрос 31. Почему проверяются пересечения секций?

Ответ. Пересекающиеся raw или virtual ranges создают неоднозначное отображение адресов и могут скрывать повреждение или намеренно конфликтующую структуру. Для parser это нарушение однозначности преобразований.

Вопрос 32. Зачем проверяется выравнивание секций?

Ответ. FileAlignment и SectionAlignment задают ожидаемую геометрию файла и образа. Нарушение может быть следствием повреждения, нестандартного генератора или намеренного затруднения анализа.

Вопрос 33. Что такое data directory?

Ответ. Это запись optional header, указывающая RVA и размер определённой таблицы, например exception directory. Проект проверяет, что указанный диапазон существует и корректно отображается.

Вопрос 34. Где находится exception directory на AMD64?

Ответ. Каталог исключений обычно указывает на массив RUNTIME_FUNCTION, часто размещённый в .pdata. Однако parser опирается на directory и RVA, а не требует конкретное имя секции.

Вопрос 35. Что обычно находится в .xdata?

Ответ. Записи UNWIND_INFO и связанные хвостовые данные: сведения об обработчиках или chained runtime function. Имя секции типично, но не является абсолютным нормативным условием.

Вопрос 36. Почему проект не доверяет именам .pdata и .xdata?

Ответ. Имена секций могут быть изменены линкером, упаковщиком или разработчиком. Основным источником истины являются адресные связи, размеры, права и структура данных.

Вопрос 37. Как parser защищается от integer overflow?

Ответ. Перед сложением смещений и размеров проверяются доступные диапазоны и условия переполнения. Чтение выполняется через bounds-safe helpers, а не через неконтролируемое приведение указателей.

Вопрос 38. Что произойдёт с PE32/x86 или ARM64?

Ответ. Файл будет отклонён как неподдерживаемая архитектура. Это сознательное ограничение: их exception/unwind-модели отличаются, поэтому нельзя безопасно применять AMD64 parser.

4. RUNTIME_FUNCTION и UNWIND_INFO

В этом разделе: 14 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 39. Что такое RUNTIME_FUNCTION?

Ответ. Для AMD64 это запись с BeginAddress, EndAddress и UnwindData. Она связывает полуоткрытый диапазон машинного кода [BeginAddress, EndAddress) с описанием раскрутки.

Вопрос 40. Почему диапазон считается полуоткрытым?

Ответ. Начальный адрес входит в функцию, конечный не входит. Такая модель исключает двойное владение граничным адресом соседними функциями и соответствует обычной адресной арифметике диапазонов.

Вопрос 41. Почему EndAddress должен быть больше BeginAddress?

Ответ. Иначе диапазон пуст или обращён назад и не может корректно описывать код функции. Это формальный структурный инвариант, а не просто эвристическое предположение.

Вопрос 42. Почему таблица должна быть упорядочена?

Ответ. Операционная система выполняет поиск записи по адресу управления. Упорядоченность обеспечивает однозначный и эффективный поиск; инверсии являются признаком несогласованности таблицы.

Вопрос 43. Почему пересекающиеся RUNTIME_FUNCTION подозрительны?

Ответ. Один адрес кода начинает соответствовать нескольким описаниям unwind. Это создаёт неоднозначность выбора метаданных и нарушает ожидаемую модель таблицы.

Вопрос 44. Что такое UNWIND_INFO?

Ответ. Переменная структура, описывающая версию, флаги, длину пролога, unwind-коды, frame register и необязательный хвост с обработчиком или chained-записью.

Вопрос 45. Что описывают unwind codes?

Ответ. Они кодируют операции, обратные действиям пролога: восстановление RSP, nonvolatile-регистров, frame pointer, XMM-регистров и некоторых специальных кадров.

Вопрос 46. Почему unwind-коды хранятся не как машинные инструкции?

Ответ. Это компактное декларативное описание необходимых обратных действий. ОС может выполнить virtual unwind без полного дизассемблирования и эмуляции каждой инструкции.

Вопрос 47. Какие версии UNWIND_INFO поддерживаются?

Ответ. Реализация декодирует используемые Windows x64 форматы V1 и V2, включая V2 epilog descriptors. Предварительный APX-формат V3 отмечается как неподдерживаемый.

Вопрос 48. Что такое chained unwind info?

Ответ. Флаг CHAININFO означает, что хвост ссылается на другую RUNTIME_FUNCTION. Это позволяет переиспользовать или связывать описания, но требует проверки глубины, циклов и границ.

Вопрос 49. Зачем ограничивать глубину цепочки?

Ответ. Повреждённый файл может создать цикл или чрезмерную цепь. Ограничение от 1 до 16 предотвращает зависание и неограниченную рекурсию, сохраняя диагностический результат.

Вопрос 50. Что такое exception handler в UNWIND_INFO?

Ответ. При соответствующих flags после выровненного массива кодов хранится RVA обработчика и language-specific data. Parser должен правильно вычислить положение хвоста и проверить адрес.

Вопрос 51. Можно ли по UNWIND_INFO полностью восстановить функцию?

Ответ. Нет. Метаданные описывают только информацию, необходимую для раскрутки и обработки исключений. Они не заменяют дизассемблирование, CFG и анализ семантики машинного кода.

Вопрос 52. Что означает fixed stack size в отчёте?

Ответ. Это размер, выводимый из статически описанных unwind-операций. Он не включает runtime-sized allocations, например alloca, поэтому является ограниченной характеристикой кадра.

5. Статический анализ и реализация parser

В этом разделе: 12 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 53. Как проходит статический анализ?

Ответ. Файл читается как недоверенный массив байтов, затем проверяются PE-заголовки и секции, отображается exception directory, читаются RUNTIME_FUNCTION, декодируется UNWIND_INFO, рассчитываются признаки и формируются отчёты.

Вопрос 54. Запускается ли файл в статическом режиме?

Ответ. Нет. Статический режим только читает байты и поэтому может безопасно работать на Linux и macOS с Windows-файлами. Риск исполнения появляется только при явном --dynamic на Windows.

Вопрос 55. Как обрабатывается повреждённая запись?

Ответ. Parser сохраняет точную причину ошибки для конкретной функции, отмечает неуспешное декодирование и продолжает анализ там, где это безопасно. Это лучше полного аварийного прекращения после первой ошибки.

Вопрос 56. Почему проект написан на C?

Ответ. C даёт прямой контроль над байтами, размерами, представлением данных и системным Windows API, а также минимальные runtime-зависимости. Риски ручного управления памятью компенсируются bounds-safe helpers, sanitizers и fuzzing.

Вопрос 57. Почему не использовать C++?

Ответ. C++ также был бы возможен. C выбран для компактного ABI, прозрачной модели владения и простоты сборки; это инженерный выбор, а не утверждение, что C объективно лучше.

Вопрос 58. Как организовано владение памятью?

Ответ. У каждого основного объекта определён владелец и функция освобождения. main.c освобождает уже созданные объекты на всех ранних путях ошибки, а sanitizer-тесты проверяют эти сценарии.

Вопрос 59. Почему parser не использует packed C-структуры поверх файла?

Ответ. Прямое отображение может нарушаться из-за выравнивания, endian assumptions, недостаточной длины и неконтролируемых указателей. Явное чтение полей позволяет проверять каждый диапазон.

Вопрос 60. Почему статический анализ кроссплатформенный?

Ответ. Формат PE — это структура байтов, для чтения которой не требуется Windows loader. Windows-зависимый код изолирован в dynamic_win.c и на других системах заменяется недоступным режимом.

Вопрос 61. Что означает strict build?

Ответ. Предупреждения компилятора включены на высоком уровне и трактуются как ошибки. Для GCC/Clang используются Wall, Wextra, Wpedantic, Wconversion, Wshadow и Wformat=2; для MSVC — W4, permissive- и WX.

Вопрос 62. Почему parser не дизассемблирует .text?

Ответ. Проект сознательно ограничен метаданными EH/Unwind. Сопоставление с машинными инструкциями повысило бы полноту, но потребовало отдельного декодера, анализа control flow и архитектурно-зависимой модели.

Вопрос 63. Что будет при неизвестном unwind opcode?

Ответ. Запись получает диагностическую аномалию или parse failure в зависимости от невозможности безопасно продолжить. Инструмент не должен молча интерпретировать неизвестные данные как корректные.

Вопрос 64. Можно ли доверять результату parser на произвольном файле?

Ответ. Можно доверять тому, что инструмент применил реализованные проверки и явно сообщил ошибки. Нельзя считать отсутствие находок доказательством полной корректности файла вне поддерживаемой модели.

6. Выбор признаков, эвристики и оценка

В этом разделе: 22 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 65. Почему выбраны именно эти признаки?

Ответ. Сначала выбраны формальные инварианты PE/COFF и Windows x64 unwind: границы, порядок, непересечение, допустимые версии и адреса. Затем добавлены интерпретируемые эвристики размещения, reuse и dominance, которые полезны реверс-инженеру и могут быть проверены вручную.

Вопрос 66. Все признаки являются эвристическими?

Ответ. Нет. Выход за границы, EndAddress не больше BeginAddress, повреждённый tail и недопустимое отображение RVA являются структурными нарушениями. Необычное имя секции, reuse и dominance — контекстные эвристики.

Вопрос 67. Какие семейства доказательств используются?

Ответ. Table integrity, unwind integrity, metadata placement, unwind reuse и handler dominance. Разделение по семьям позволяет отличать несколько независимых подтверждений от множества однотипных симптомов.

Вопрос 68. Почему выход за границы имеет высокий вес?

Ответ. Он означает, что адрес или размер не может быть корректно отображён в поддерживаемой модели файла. Это сильнее, чем необычное, но всё ещё допустимое размещение.

Вопрос 69. Почему проверяется writable metadata?

Ответ. Компиляторные unwind-метаданные обычно являются неизменяемыми данными. Записываемое размещение допускает модификацию во время исполнения и потому требует внимания, особенно вместе с writable handler target.

Осторожно: Writable не означает автоматически malicious.

Вопрос 70. Почему UNWIND_INFO в executable section считается подозрительным?

Ответ. Метаданные являются данными, а executable permission обычно предназначено для кода. Такое совмещение увеличивает поверхность интерпретации и нетипично для обычной компиляторной раскладки, хотя возможно в нестандартных сборках.

Вопрос 71. Почему handler в non-executable section подозрителен?

Ответ. Handler RVA должен указывать на исполняемую логику. Если адрес не принадлежит executable section или вообще не отображается, возникает противоречие между ролью адреса и правами памяти.

Вопрос 72. Почему анализируется reuse одного UnwindInfoRVA?

Ответ. Сильная централизация описаний может быть нормальной оптимизацией, но также встречается при нестандартных dispatcher-based layouts. Поэтому умеренный reuse учитывается только при наличии другого сигнала, а сильный — на больших таблицах.

Вопрос 73. Почему handler dominance не всегда является аномалией?

Ответ. Runtime или библиотека могут законно использовать общий обработчик для множества функций. Реализация подавляет этот сигнал, если обработчик находится в нормальной executable code-like section и других проблем нет.

Вопрос 74. Откуда взялись пороги 1%, 5%, 30% и 80%?

Ответ. Это консервативные инженерные defaults, разделяющие единичный дефект и массовый паттерн. Они зафиксированы в коде и документации, но не заявляются статистически оптимальными для всех компиляторов и защит.

Осторожно: Нельзя называть их результатом машинного обучения или ROC-калибровки.

Вопрос 75. Откуда взялись веса +10, +20 и +25?

Ответ. Вес отражает инженерную силу и устойчивость сигнала: формальные нарушения получают больший вклад, контекстные признаки — меньший. Итоговая шкала предназначена для triage и сравнения сигналов, а не для вычисления вероятности.

Вопрос 76. Почему score ограничен 0–100?

Ответ. Это удобная нормализованная шкала для отчёта и интерфейса. При суммировании используется saturation, поэтому значение показывает уровень приоритета, а не число независимых нарушений.

Вопрос 77. Что означает suspicion?

Ответ. Это score, делённый на 100. Поле удобно для машинной обработки, но не является статистической вероятностью вредоносности.

Вопрос 78. Что означает confidence?

Ответ. Это инженерная уверенность в интерпретации совокупности сигналов. Она растёт при нескольких семьях и сильных нарушениях и уменьшается при единичном сигнале, маленьком образце или placement-only случае.

Вопрос 79. Как снижаются ложные срабатывания?

Ответ. Используются weakening rules: single_signal_only, small_sample_size, placement_only_signal и single_family_only. Умеренные reuse/dominance-сигналы требуют контекста, а итоговый отчёт рекомендует ручную проверку.

Вопрос 80. Почему комбинации признаков получают дополнительный вес?

Ответ. Два согласованных сигнала труднее объяснить случайностью. Например, writable metadata вместе с writable handler target сильнее каждого признака по отдельности.

Вопрос 81. Может ли нормальный файл получить ненулевой score?

Ответ. Да, особенно при нестандартном toolchain или layout. Поэтому score используется как приоритет ручного анализа, а не как бинарный приговор.

Вопрос 82. Может ли вредоносный файл получить нулевой score?

Ответ. Да. Вредоносная логика может не затрагивать EH/Unwind-метаданные или сохранять их корректными. Инструмент не является универсальным детектором malware.

Вопрос 83. Как доказать обоснованность признаков?

Ответ. Для каждого finding указаны семейство, тяжесть, вес, условие и возможное ослабление в evidence_matrix.md. Сильные признаки выводятся из структурных инвариантов, а слабые явно маркируются как эвристические и проверяются на benign-корпусе.

Вопрос 84. Почему не использованы entropy, imports и сигнатуры?

Ответ. Они относятся к общему анализу упаковки и вредоносности. Проект намеренно сохраняет узкую предметную область EH/Unwind, чтобы выводы были объяснимыми и непосредственно связаны с исследуемыми структурами.

Вопрос 85. Почему не применено машинное обучение?

Ответ. Для ML нужен большой размеченный и репрезентативный корпус с контролем смещений. В данной работе важнее интерпретируемость: каждый результат можно связать с конкретным RVA, структурой и нарушенным правилом.

Вопрос 86. Что означает requires_manual_review?

Ответ. Это явный флаг, что совокупность признаков или применённое ослабление требует экспертной проверки. Он не утверждает наличие атаки, а управляет рабочим процессом аналитика.

7. Динамический анализ на Windows

В этом разделе: 18 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 87. Зачем нужна динамика, если есть статический анализ?

Ответ. Статика показывает, что записано в файле. Динамика подтверждает, какие обработчики и API реально активируются при конкретном запуске, и позволяет коррелировать потенциальную структуру с наблюдаемым поведением.

Вопрос 88. Как запускается динамический режим?

Ответ. На Windows x64 программа создаёт целевой EXE под Windows Debug API, обрабатывает debug events, устанавливает контролируемые software breakpoints и сохраняет структурированную временную линию.

Вопрос 89. Какие события собираются?

Ответ. Исключения первого и второго шанса, регистрации и hits VEH/VCH/UEF, hits статически известных handlers, вызовы RtlAddFunctionTable и RtlInstallFunctionTableCallback, exit status, timeout и ограниченная RIP trace.

Вопрос 90. Что такое VEH?

Ответ. Vectored Exception Handler — обработчик, регистрируемый на уровне процесса и вызываемый в цепочке vectored exception handling независимо от конкретного stack frame.

Вопрос 91. Что такое VCH?

Ответ. Vectored Continue Handler — обработчик этапа продолжения после обработки исключения. Он отличается ролью от VEH и регистрируется отдельным Windows API.

Вопрос 92. Что такое UEF?

Ответ. Unhandled Exception Filter — фильтр верхнего уровня для необработанных исключений процесса, устанавливаемый SetUnhandledExceptionFilter.

Вопрос 93. Зачем отслеживается RtlAddFunctionTable?

Ответ. Этот API регистрирует runtime function table, отсутствующую в статическом exception directory. Он используется JIT и другими динамическими системами, а также важен при анализе нестандартной runtime-генерации кода.

Вопрос 94. Как работают software breakpoints?

Ответ. В целевой процесс временно записывается INT3. При срабатывании collector восстанавливает исходный байт, корректирует RIP, включает trap flag для single-step и затем переустанавливает breakpoint.

Вопрос 95. Не повреждает ли breakpoint программу?

Ответ. Он временно изменяет один байт памяти отлаживаемого процесса и затем восстанавливает его. Это стандартная техника отладки, но она всё равно влияет на timing и может обнаруживаться anti-debug логикой.

Вопрос 96. Что такое RIP trace?

Ответ. После первого подтверждённого handler hit collector сохраняет ограниченное число адресов исполнения через single-step. В проверенной конфигурации бюджет ограничен 32 адресами, чтобы отчёт и время не росли бесконтрольно.

Вопрос 97. Почему trace не является полной трассировкой?

Ответ. Она короткая, начинается только после определённого события и не записывает все инструкции процесса. Это локальное свидетельство потока управления, а не полноценный execution tracer.

Вопрос 98. Что происходит при timeout?

Ответ. Collector отмечает timed_out, инициирует завершение целевого процесса, дренирует оставшиеся debug events в ограниченном cleanup window и формирует согласованный отчёт вместо зависания.

Вопрос 99. Почему динамика работает только на Windows?

Ответ. Она использует Windows Debug API, контексты Windows threads и соглашения PE image loader. Linux и macOS могут читать PE статически, но не предоставляют эквивалентную Windows runtime-среду.

Вопрос 100. Можно ли динамически анализировать DLL и PYD?

Ответ. Не напрямую текущим runner, потому что CreateProcess запускает EXE. Для DLL/PYD нужен отдельный host process, который загрузит библиотеку и вызовет нужную точку входа.

Вопрос 101. Безопасно ли запускать неизвестный файл?

Ответ. Только в изолированной Windows VM без секретов, shared folders и лишней сети. Collector является отладчиком и сборщиком свидетельств, а не malware sandbox.

Вопрос 102. Может ли anti-debug повлиять на результат?

Ответ. Да. Цель может менять поведение, завершаться или скрывать ветви при обнаружении отладчика. Поэтому runtime evidence является наблюдением конкретного запуска, а не полной моделью всех возможных путей.

Вопрос 103. Что значит static/dynamic correlation?

Ответ. Проект связывает статические handler/placement-признаки с runtime hits или динамическими function tables. Это консервативная корреляция на уровне классов событий, не per-instruction proof graph.

Вопрос 104. Почему dynamic ok может быть false?

Ответ. Причиной может быть невозможность запуска, timeout cleanup failure, ошибка доступа к context/process memory, breakpoint re-arm failure или нарушение продолжения debug event. Такие случаи явно отражаются в отчёте и exit code.

8. Отчёты, JSON и Ghidra

В этом разделе: 12 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 105. Какие форматы отчётов поддерживаются?

Ответ. Консоль, JSON, автономный HTML, общий CSV и Ghidra CSV. Несколько форматов можно создать за один запуск.

Вопрос 106. Почему JSON является основным интеграционным контрактом?

Ответ. Он содержит полную структурированную модель, версию schema, provenance, hash, functions, findings, dynamic events и assessment. Backend может валидировать и сохранять его без разбора консольного текста.

Вопрос 107. Зачем JSON Schema?

Ответ. Schema фиксирует типы, обязательные поля, диапазоны и структуру отчёта. Это снижает риск незаметного нарушения контракта при развитии ядра.

Вопрос 108. Чем семантическая проверка отличается от Schema?

Ответ. Schema проверяет форму данных. Семантический validator пересчитывает связи: размеры диапазонов, счётчики, согласованность assessment, trace budget и другие cross-field invariants.

Вопрос 109. Что содержит HTML-отчёт?

Ответ. Самодостаточный интерактивный dashboard с общей оценкой, находками, таблицами функций и динамическими событиями. Он не загружает внешние scripts, styles или fonts.

Вопрос 110. Как защищён HTML от внедрения кода?

Ответ. Строки, полученные из входного файла и metadata, экранируются. Отчёт содержит ограничительную offline Content Security Policy и не обращается к сети.

Вопрос 111. Для чего нужен обычный CSV?

Ответ. Он удобен для плоского анализа функций в таблицах, фильтрации, статистики и последующей обработки внешними инструментами.

Вопрос 112. Что делает Ghidra importer?

Ответ. Он читает специальный CSV, проверяет поля и адреса, создаёт или дополняет функции, объединяет tagged comments и ставит warning bookmarks для parse failures и аномалий.

Вопрос 113. Почему Ghidra не встроена прямо в ядро?

Ответ. Ядро остаётся независимым CLI-инструментом, а Ghidra integration реализована отдельным adapter script. Это уменьшает зависимости и позволяет использовать отчёты без Ghidra.

Вопрос 114. Как обеспечить связь отчёта с исходным файлом?

Ответ. В report metadata сохраняется SHA-256 исследуемого файла, версия инструмента, schema version, platform, compiler и время. Это поддерживает воспроизводимость и chain of custody.

Вопрос 115. Можно ли подключить базу данных и frontend?

Ответ. Да. Backend принимает бинарник, вызывает CLI, проверяет JSON Schema и сохраняет JSON и hash в БД. Frontend работает через API backend; менять алгоритмическое ядро для базового сценария не требуется.

Вопрос 116. Почему запись отчёта атомарная?

Ответ. Writer сначала создаёт временный sibling-файл и затем заменяет итоговый. Если процесс прервётся, ранее существовавший корректный отчёт не должен быть частично уничтожен.

9. Архитектура и инженерные решения

В этом разделе: 12 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 117. Какая архитектура у проекта?

Ответ. Модульный монолит: библиотечное ядро ehua_core и отдельный CLI executable. Модули разделены по ответственности, но поставляются как единый компактный инструмент.

Вопрос 118. Почему не микросервисы?

Ответ. Для локального анализатора микросервисы добавили бы сеть, deployment и распределённые отказы без пользы для алгоритма. Сервисный слой можно построить поверх стабильного CLI/JSON, не превращая ядро в распределённую систему.

Вопрос 119. Чем CLI отличается от ядра?

Ответ. Ядро содержит parser, decoder, heuristics, dynamic collector и serializers. CLI разбирает аргументы, вызывает этапы в нужном порядке, управляет файлами и возвращает exit code.

Вопрос 120. Почему ядро выделено в static library?

Ответ. Это уменьшает связанность main.c с реализацией и позволяет повторно использовать функции в tests или будущем embedding API. При этом релиз остаётся одним executable.

Вопрос 121. Какие основные модули существуют?

Ответ. util — безопасные байты и SHA-256; pe — заголовки и RVA; unwind — декодирование; heuristics — findings и verdict; dynamic_win — Windows events; report — serializers; app_info — provenance; main — orchestration.

Вопрос 122. Как обеспечивается расширяемость?

Ответ. Новые признаки добавляются как structured findings и документируются в evidence matrix; новые report fields требуют schema decision и tests; новые dynamic hooks оформляются как breakpoint roles и events.

Вопрос 123. Можно ли добавить ARM64?

Ответ. Да, но как отдельную architecture-specific модель parser/unwind, а не через ослабление AMD64 assumptions. Формат exception metadata и правила unwind для ARM64 отличаются.

Вопрос 124. Можно ли сделать библиотечный API?

Ответ. Да. Большая часть логики уже находится в ehua_core. Для публичного API потребуется стабилизировать ownership, error model, ABI/versioning и потоковую передачу результатов.

Вопрос 125. Почему используется CMake?

Ответ. Он поддерживает несколько компиляторов и ОС, targets, install rules, CTest, sanitizer/coverage presets и CPack. Это соответствует кроссплатформенному статическому ядру и Windows-специфичной динамике.

Вопрос 126. Какова сложность основных этапов?

Ответ. Последовательный разбор таблицы близок к O(n), а проверка overlaps выполняется после сортировки — O(n log n). Hash maps используются для подсчёта reuse и dominance без квадратичного сравнения всех записей.

Вопрос 127. Почему ограничены массивы findings и mitigations?

Ответ. Отчёт агрегирует классы проблем, а не хранит бесконечный поток одинаковых сообщений. Ограничение делает модель памяти предсказуемой, при этом affected_count и representative RVA сохраняют масштаб и пример.

Вопрос 128. Как проект обрабатывает ошибки?

Ответ. CLI различает invalid command line, input/analysis/output failure и успешное выполнение. Ошибки parser сохраняются максимально локально, а невозможность выполнить требуемый этап приводит к ненулевому exit code.

10. Тестирование, валидация и показатели

В этом разделе: 18 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 129. Сколько автоматических тестов проходит?

Ответ. Итоговый release preset проходит 16 из 16 CTest tests. В набор входят unit, regression, schema/semantics, CLI contract, release metadata/hygiene и mutation fuzz.

Вопрос 130. Что проверяют unit tests?

Ответ. Декодирование unwind, эвристические комбинации и mitigations, edge cases ядра, JSON/HTML serialization и matrix report. Они изолируют конкретные функции и инварианты.

Вопрос 131. Что такое regression samples?

Ответ. Это шесть воспроизводимых синтетических PE-файлов с заранее известными ожидаемыми признаками: нормальные и специально повреждённые варианты. Manifest задаёт ожидаемый score, verdict и findings.

Вопрос 132. Зачем нужны синтетические файлы?

Ответ. Они дают управляемую причинность: известно, какая именно структура изменена и какой сигнал должен возникнуть. Реальный файл редко позволяет столь однозначно связать дефект с ожидаемым результатом.

Вопрос 133. Тестировался ли проект на реальных файлах?

Ответ. Да. Статический benign-корпус содержит 106 PE32+ файлов из известных программных поставок. Получено 106/106 валидных отчётов и разобрано 174 795 RUNTIME_FUNCTION.

Вопрос 134. Что означает ноль аномалий на benign-корпусе?

Ответ. На конкретном корпусе не было parse failures, structural unwind anomalies и ненулевых scores. Это положительный результат по ложным срабатываниям, но не универсальная гарантия для всех легитимных toolchains.

Вопрос 135. Что такое mutation testing в проекте?

Ответ. Детерминированно изменяются байты и структуры PE, после чего parser запускается с ограничениями времени. Проверяется отсутствие crashes/hangs и корректность поведения отчёта.

Вопрос 136. Каков результат расширенного mutation run?

Ответ. Тысяча мутаций обработана без аварий и некорректных отчётов. Это проверка устойчивости parser, а не доказательство отсутствия всех возможных ошибок.

Вопрос 137. Что проверяют ASan и UBSan?

Ответ. AddressSanitizer ищет выходы за границы, use-after-free и связанные ошибки памяти. UndefinedBehaviorSanitizer выявляет ряд неопределённых операций языка C.

Вопрос 138. Что такое libFuzzer?

Ответ. Coverage-guided fuzzer генерирует входы, ориентируясь на новые ветви выполнения. В проекте есть parser harness и Linux CI campaign; дополнительно используется воспроизводимый mutation runner.

Вопрос 139. Каково покрытие кода?

Ответ. Для кроссплатформенного ядра без Windows-only dynamic branch зафиксировано 86,3% строк, 96,2% функций и 62,8% ветвей.

Вопрос 140. Почему branch coverage ниже line coverage?

Ответ. Каждая строка может содержать несколько логических исходов, особенно в error handling и проверках повреждённых данных. Покрыть все комбинации ветвей сложнее, чем выполнить строку хотя бы один раз.

Вопрос 141. Что проверяет Clang Static Analyzer?

Ответ. Он выполняет статический поиск потенциальных путей утечки, use-after-free, null dereference и других дефектов. На проверенной ревизии он не сообщил замечаний.

Вопрос 142. Как проверена Windows-динамика?

Ответ. На Windows x64 с MinGW-w64 валидированы 124 из 124 JSON-отчётов. Запущено 86 dynamic sessions: 54 завершились, 32 контролируемо достигли timeout; четыре focused smoke scenarios прошли.

Вопрос 143. Почему timeout-сессии не считаются провалом?

Ответ. Некоторые GUI или долгоживущие программы не завершаются самостоятельно. Если collector соблюдает deadline, корректно завершает цель, сохраняет timed_out и валидный отчёт, это ожидаемый контролируемый исход.

Вопрос 144. Что означает 4/4 dynamic smoke?

Ответ. Отдельные синтетические программы подтвердили регистрацию и hit VEH, VCH, UEF, а также наблюдение обоих runtime function-table APIs. Это целевые проверки ключевых ветвей collector.

Вопрос 145. Проверялись ли потерянные события?

Ответ. В зафиксированной Windows-валидации dropped events равны нулю. Это относится к конкретным тестовым прогонам и не является гарантией при любой нагрузке.

Вопрос 146. Почему результаты можно считать воспроизводимыми?

Ответ. Зафиксированы version, source tree, analyzer SHA-256, schema, scripts, input hashes, отчёты и validators. Release bundles генерируются детерминированно и повторно дали одинаковые SHA-256.

11. Платформы, безопасность и релиз

В этом разделе: 12 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 147. На каких ОС работает проект?

Ответ. Статический анализ собирается и работает на Windows, Linux и macOS. Динамический режим доступен только на Windows x64.

Вопрос 148. Можно ли на Linux анализировать Windows EXE?

Ответ. Да, статически. PE-файл читается как набор байтов; запуск Windows API не требуется.

Вопрос 149. Можно ли на Linux выполнить Windows-динамику?

Ответ. Не в нативном режиме текущего проекта. Для достоверной Windows Debug API сессии нужна Windows x64, обычно физическая система или изолированная VM.

Вопрос 150. Зачем версия для macOS?

Ответ. Она позволяет исследователю на Apple Silicon выполнять статический анализ Windows PE32+ и формировать отчёты без переноса файла на Windows. Для runtime validation всё равно нужна Windows.

Вопрос 151. Почему релиз называется 1.0.0-rc1?

Ответ. Функциональное ядро завершено, MinGW Windows validation выполнена, но для формального promotion до 1.0.0 предусмотрен независимый MSVC gate на финальной ревизии.

Вопрос 152. Само ядро готово?

Ответ. Да, для заявленной области оно функционально завершено: parser, decoder, heuristics, reports и dynamic collector реализованы и проверены. RC-статус относится к формальному multi-toolchain release gate, а не к отсутствию основной функции.

Вопрос 153. Какие релизные пакеты существуют?

Ответ. Windows x64 — готовый инструмент; Darwin arm64 — статический инструмент для Apple Silicon; source — исходники и тесты; evidence — доказательная база Windows-проверки. SHA256SUMS проверяет целостность.

Вопрос 154. Почему в клиентский архив не входит весь real corpus?

Ответ. Корпус нужен для инженерной валидации, занимает много места и содержит сторонние файлы с отдельными лицензионными условиями. Обычная поставка включает инструмент, документацию и проверяемые evidence artifacts.

Вопрос 155. Какие данные отчёта могут быть чувствительными?

Ответ. Пути файлов, имена модулей, section names, addresses и runtime metadata. Аргументы после -- редактируются в metadata, но отчёт всё равно следует хранить как аналитический артефакт.

Вопрос 156. Почему рекомендуются ASCII paths на Windows?

Ответ. Проверенная dynamic revision использует narrow Win32 API CreateProcessA. Для максимально воспроизводимого запуска избегаются зависимости от локальной code page; переход на wide API является возможным улучшением.

Вопрос 157. Какая лицензия у проекта?

Ответ. MIT. Она разрешает использование, изменение и распространение при сохранении уведомления об авторских правах и текста лицензии.

Вопрос 158. Как проверить целостность поставки?

Ответ. Сравнить SHA-256 архивов с SHA256SUMS.txt, а Windows executable — с VERIFIED_WINDOWS_BUILD.json. Затем распаковать архив и выполнить documented smoke/build tests.

12. Жёсткие и провокационные вопросы комиссии

В этом разделе: 24 вопросов. Формулировки согласованы с текущим кодом, отчётами и паспортом качества.

Вопрос 159. Разве это не просто parser?

Ответ. Parser является основой, но продукт включает структурную validation, cross-record analysis, интерпретируемые findings, score/confidence/mitigations, динамический collector, schema-validated reports, Ghidra integration и release/test infrastructure.

Вопрос 160. Где доказательство, что аномалия связана с атакой?

Ответ. Проект не утверждает такую причинность. Он обнаруживает отклонение от структурной или типичной модели и предоставляет evidence для эксперта; связь с атакой устанавливается последующим анализом кода, поведения и контекста.

Осторожно: Лучший ответ — признать границу доказательства, а не защищать чрезмерное утверждение.

Вопрос 161. Почему score можно считать объективным?

Ответ. Его нельзя считать универсально объективной вероятностью. Он объективен только в смысле детерминированного применения опубликованной матрицы правил к одному и тому же входу.

Вопрос 162. Почему ваши веса не взяты из статистики?

Ответ. Работа использует инженерную интерпретируемую модель из-за отсутствия большого качественно размеченного EH/Unwind-корпуса. Это ограничение честно зафиксировано; проект предоставляет score sweep для будущей калибровки.

Вопрос 163. Тогда можно ли вообще доверять эвристикам?

Ответ. Да, как инструменту triage. Сильные findings основаны на формальных нарушениях, слабые имеют mitigations, а отчёт показывает не только число, но и причины, locations и manual validation steps.

Вопрос 164. Почему 106 файлов достаточно?

Ответ. Этого достаточно для подтверждения совместимости с выбранным benign-набором, но недостаточно для универсальной статистической оценки. Поэтому в работе не заявляются sensitivity, specificity или ROC-level качество.

Вопрос 165. Где набор реальных вредоносных файлов?

Ответ. Проект не поставляет malware corpus по причинам безопасности, воспроизводимости и лицензирования. Корректность отдельных признаков проверяется синтетическими targeted samples, а benign corpus оценивает совместимость и ложные срабатывания.

Вопрос 166. Без malware-корпуса как доказать полезность?

Ответ. Полезность состоит не только в классификации malware: инструмент обнаруживает malformed metadata, помогает восстановить функции, локализует handlers и сокращает ручной анализ. Targeted samples подтверждают причинную работу правил.

Вопрос 167. Почему не сравнили точность с Ghidra?

Ответ. Ghidra — платформа статического анализа, а не прямой бинарный классификатор EH/Unwind findings. Проект дополняет её: генерирует формализованный анализ и импортирует результаты в Ghidra для ручного исследования.

Вопрос 168. Почему не использовать dumpbin или WinDbg?

Ответ. Они полезны для просмотра и отладки, но проект решает другую задачу: пакетная validation, агрегированные эвристики, строгий JSON-контракт, воспроизводимые tests и одновременная статико-динамическая отчётность.

Вопрос 169. Почему HTML, если пользователи работают в терминале?

Ответ. CLI и JSON остаются основными профессиональными интерфейсами. HTML нужен для навигации по большим отчётам, демонстрации заказчику и защиты; он не является обязательным для автоматизации.

Вопрос 170. Почему нет GUI?

Ответ. Целевая аудитория может работать через CLI, Ghidra и JSON. Отделение GUI сохраняет ядро компактным; полноценный frontend можно строить поверх backend без изменения алгоритма.

Вопрос 171. Почему нет базы данных?

Ответ. БД относится к системе хранения и сравнения результатов, а не к алгоритмическому ядру. JSON содержит stable identifiers и hash, поэтому backend может сохранить результат без внедрения SQL-зависимости в parser.

Вопрос 172. Не является ли динамический режим опасным?

Ответ. Да, он пересекает trust boundary и запускает цель. Поэтому документация требует isolated VM; проект не позиционируется как sandbox и не обещает containment.

Вопрос 173. Почему динамика не покрывает все пути?

Ответ. Один запуск зависит от входов, окружения и anti-debug. Для полного path exploration потребовались бы symbolic execution, instrumentation или множественные сценарии, что выходит за область работы.

Вопрос 174. Что если файл сам изменяет unwind-таблицы?

Ответ. Статика зафиксирует исходное состояние. Dynamic collector наблюдает registrations через function-table APIs, но не реконструирует каждую запись и каждую произвольную memory write; это направление развития.

Вопрос 175. Что если файл корректный структурно, но вредоносный?

Ответ. Он может не получить findings. Проект анализирует EH/Unwind anomalies, а не семантику всей программы, сетевое поведение или malicious intent.

Вопрос 176. Что если protector легально использует необычные структуры?

Ответ. Он может вызвать эвристические findings. Именно поэтому отчёт разделяет signal, confidence и manual review, а не объявляет файл вредоносным.

Вопрос 177. Можно ли обойти анализатор?

Ответ. Любой ограниченный анализ можно обойти, сохранив корректные метаданные, переместив логику в неподдерживаемую область или используя anti-debug. Цель инструмента — повысить наблюдаемость выбранного класса структур, а не обеспечить абсолютное обнаружение.

Вопрос 178. Почему проект можно считать завершённым?

Ответ. Все функции заявленной области реализованы, имеют отчётный контракт и проверены tests/corpora. Оставшиеся идеи — disassembly correlation, DLL host, wide Windows API, larger calibration corpus — являются расширениями, а не отсутствующими базовыми функциями.

Вопрос 179. Какое главное ограничение работы?

Ответ. Отсутствие большой статистической калибровки на разнообразном размеченном corpus и отсутствие сопоставления unwind metadata с реальными инструкциями пролога. Эти ограничения прямо обозначены и задают следующий этап развития.

Вопрос 180. Какое главное достоинство работы?

Ответ. Интерпретируемость и проверяемость: каждый вывод связан с формальным правилом или явно обозначенной эвристикой, конкретным адресом, structured report и воспроизводимой validation procedure.

Вопрос 181. Что бы вы сделали следующим этапом?

Ответ. Добавил бы instruction-level verification прологов, Unicode Windows paths, DLL/PYD host runner, расширенный размеченный corpus, ROC/threshold calibration и сервисный backend для хранения и сравнения отчётов.

Вопрос 182. Как ответить, если вы не знаете точную деталь?

Ответ. Нужно отделить подтверждённый факт от предположения: сказать, что конкретная деталь не проверялась, назвать известную границу реализации и объяснить, где это можно установить по коду, report или документации.

Осторожно: Не придумывайте значения полей стандарта и результаты тестов.

Финальная памятка перед входом на защиту

Что нужно помнить без подсказки

- Проект: анализатор PE32+ AMD64 EH/Unwind-метаданных.
- Архитектура: модульный монолит, библиотечное ядро + CLI.
- Статика: Windows, Linux, macOS. Динамика: только Windows x64.
- Основные структуры: exception directory, RUNTIME_FUNCTION, UNWIND_INFO.
- Результат: findings, score, confidence, mitigations, JSON/HTML/CSV и Ghidra.
- Score — triage, не вероятность и не доказательство malware.
- 16/16 CTest; 1000 mutations; 106/106 benign; 124/124 Windows reports.
- Версия 1.0.0-rc1; независимый MSVC gate нужен для формального 1.0.0.

Фразы, которых следует избегать

Неверно	Корректно
«Программа определяет вредоносность».	«Программа выявляет EH/Unwind-аномалии и приоритизирует ручной анализ».
«Score — вероятность атаки».	«Score — детерминированная инженерная шкала подозрительности».
«Динамика показывает всё поведение».	«Динамика фиксирует ограниченный набор событий конкретного запуска».
«На корпусе доказана абсолютная точность».	«Корпус подтвердил совместимость и отсутствие findings на выбранных benign-файлах».
«Проект полностью неуязвим».	«Проект прошёл заявленные quality gates; абсолютная безошибочность не утверждается».

Главный принцип ответа: не защищать проект чрезмерными обещаниями. Сильная защита строится на точном понимании границ, воспроизводимых фактах и способности объяснить, почему каждый вывод следует из конкретных данных.

13. Аналоги, конкуренты и рыночное позиционирование

Исследование публично доступных зарубежных и российских инструментов, которые пересекаются с EH/Unwind Analyzer по разбору PE, x64 exception/unwind, поиску аномалий, reverse engineering или динамической проверке.

Главный вывод

В исследованном наборе официальных документов и первичных репозиториев не найден готовый инструмент, который повторяет всю комбинацию проекта: строгий разбор RUNTIME_FUNCTION/UNWIND_INFO, объяснимые cross-record эвристики, score и confidence, ослабляющие правила, автономные JSON/HTML/CSV-отчёты и корреляцию с Windows runtime events. Однако отдельные части этой комбинации реализованы у более зрелых и гораздо более широких продуктов. Поэтому корректное позиционирование — специализированный аналитический модуль, а не замена IDA, Ghidra, LIEF или WinDbg.

Дата среза: 9 июня 2026 года.

Версия сравниваемого проекта: 1.0.0-rc1. Состояние функций сверено с README.md, docs/ARCHITECTURE.md, docs/PRODUCT_READINESS.md, docs/evidence_matrix.md и исходным кодом.

Ограничение: фраза «весь интернет» не может означать буквально все существующие закрытые продукты и внутренние разработки. Исследование охватывает репрезентативный набор публично документированных решений и не утверждает, что неизвестных или непубличных аналогов не существует.

13.1. Методика поиска и правила честного сравнения

Поиск выполнен по официальной документации Microsoft, страницам производителей, официальным репозиториям и документации проектов. Блоги-агрегаторы, рекламные пересказы и случайные списки «лучших PE tools» не использовались как основание для функциональных утверждений.

Сравнение проводилось в четыре шага:

• **Функциональное пересечение:** умеет ли продукт читать PE exception directory, RUNTIME_FUNCTION и UNWIND_INFO.

• **Аналитическая глубина:** только показывает поля или выявляет нарушения между записями, секциями, правами памяти и обработчиками.

• **Эксплуатационная форма:** GUI, CLI, библиотека, API, форматы отчётов, автоматизация, работа без облака.

• **Доказательность:** объясняются ли причины вывода, есть ли score, confidence, provenance и ручные шаги проверки.

Что считается фактом
Фактом считается только возможность, явно заявленная в официальной документации или видимая в первичном репозитории. Формулировки «не найдено», «не заявлено» и «проект выглядит сильнее» являются аналитическими выводами, а не доказательством полного отсутствия функции в любом плагине или закрытой редакции.

Критерии сравнения

Критерий	Что именно оценивается
Фокус EH/Unwind	Насколько предметная область является центральной, а не побочной.
Глубина структуры	Границы, сортировка, overlaps, chains, handlers, opcodes, placement.
Объяснимость	Коды findings, причины, affected count, representative RVA, manual steps.
Автоматизация	CLI/API, стабильная schema, пакетная обработка, CI/БД-интеграция.
Динамика	Фактические exceptions, handler hits, registrations, dynamic function tables.
Анализ кода	Disassembly, decompilation, CFG, data flow, сопоставление с прологом.
Зрелость	История проекта, ecosystem, plugins, поддержка форматов и архитектур.

13.2. Карта конкурентного поля

Один общий список конкурентов вводит в заблуждение. Ниже инструменты разделены по тому, какую часть рабочего процесса они действительно перекрывают.

Уровень	Инструменты	Конкурируют за
Ближайшие технические аналоги	LIEF, llvm-readobj, Dissect PE	Декодирование exception/unwind structures и программный доступ к ним.
Широкие PE triage/viewers	pestudio, PE-bear, 4n4lDetector, pefile, readpe, PE Tree, XPEViewer	Первичный осмотр PE, indicators, секции, imports, packer и структурные свойства.
Reverse-engineering platforms	IDA, Ghidra, Binary Ninja, radare2	Время аналитика, интерактивное исследование кода, функций и control flow.
Dynamic/debugging	WinDbg, x64dbg, встроенные debuggers IDA/Binary Ninja	Наблюдение реального выполнения, exceptions, breakpoints и stack unwinding.
Смежная security-аналитика	BinSkim, capa, Detect It Easy	Security posture, capabilities, compiler/packer identification и triage.

Кто является самым близким конкурентом

LIEF ближе всего по глубине программного представления PE x64 exceptions. **pestudio и 4n4lDetector** ближе по идее автоматического PE triage и indicators. **IDA/Ghidra** ближе по конечной аудитории. **WinDbg/x64dbg** ближе по динамическому подтверждению. Но ни один из них нельзя честно назвать полным прямым аналогом всей системы.

Корректная рыночная формулировка

EH/Unwind Analyzer — узкоспециализированный explainable triage-инструмент для Windows PE32+ AMD64 exception/unwind metadata. Он располагается между библиотекой форматного разбора, PE anomaly scanner и reverse-engineering workflow: автоматически выделяет подозрительные места и передаёт их человеку или внешней платформе.

13.3. Сводная функциональная матрица

Обозначения: **++** — центральная и развитая возможность; **+** — имеется или достигается штатными средствами; **±** — частично, косвенно или требует дополнительной настройки; **—** — в исследованной официальной документации не заявлено как готовая функция.

Инструмент	EH/UW parse	Аномалии	Code RE	Dynamic	Schema/CLI	Фокус
EH/Unwind Analyzer	++	++	—	+	++	Узкий
LIEF	++	±	—	—	++	Библиотека
llvm-readobj	+	—	—	—	+	Инспекция
Dissect PE	+	—	—	—	++	Библиотека
pestudio	±	+	—	—	±	PE triage
PE-bear	±	±	±	—	—	PE viewer
4n4lDetector	±	+	±	—	±	PE anomalies
IDA Pro	+	±	++	+	+	Полный RE
Ghidra	+	±	++	+	+	Полный RE
Binary Ninja	+	±	++	+	++	Полный RE/API
radare2	±	±	++	+	++	RE framework
WinDbg	+	—	++	++	+	Debugger
x64dbg	±	—	++	++	+	Debugger
BinSkim	—	+	—	—	++	Security lint
capa	—	+	++	—	++	Capabilities
Detect It Easy	—	+	±	—	+	Packer/type

Как читать таблицу

Матрица сравнивает готовые функции «из коробки», а не теоретическую возможность написать plugin. IDA, Ghidra, Binary Ninja и radare2 программируемы, поэтому почти любой узкий анализ можно реализовать расширением. Это не отменяет ценность отдельного готового инструмента, но снижает технологический барьер для копирования идеи.

13.4. Ближайшие технические аналоги

LIEF

Класс: кроссплатформенная библиотека C++ с Python/Rust bindings

Что подтверждено источниками. Документация описывает PE exception information, RuntimeFunctionX64, unwind opcodes, chained entries и связанные структуры. В версии 0.17.0 поддержка PE exceptions была существенно расширена.

Где аналог сильнее. Гораздо шире поддержка форматов, архитектур и операций чтения/изменения; зрелая API-экосистема; удобна как фундамент собственного продукта.

Где EH/Unwind Analyzer сильнее или удобнее. Проект выдаёт готовые findings, evidence families, score, confidence, mitigations, manual validation steps, автономный HTML и Windows runtime evidence. Для LIEF такой предметный аналитический слой требуется писать отдельно.

Итог. Самый близкий конкурент на уровне parser/model, но скорее потенциальный backend или зависимость, чем законченный конкурент аналитического workflow.

Источники: [S5], [S6]

llvm-readobj --unwind

Класс: официальная CLI-утилита LLVM для низкоуровневого отображения object-file structures

Что подтверждено источниками. Command Guide документирует опцию --unwind для показа unwind information.

Где аналог сильнее. Надёжная часть LLVM toolchain, удобна для компиляторной диагностики и проверки сгенерированных объектов; узнаваемый стандартный CLI.

Где EH/Unwind Analyzer сильнее или удобнее. Не формирует специализированную модель аномалий по всему образу, score/confidence, placement/reuse/dominance findings, динамическую корреляцию и analyst report.

Итог. Сильный baseline: комиссия может спросить, почему недостаточно llvm-readobj. Ответ — он показывает данные, а проект интерпретирует взаимосвязи и приоритизирует проверку.

Источники: [S7]

Dissect PE

Класс: Python-библиотека forensic tooling

Что подтверждено источниками. Документация предоставляет программный доступ к AMD64 exception directory и runtime/unwind records.

Где аналог сильнее. Удобна для Python forensic pipelines, быстро интегрируется с другими компонентами экосистемы Dissect.

Где EH/Unwind Analyzer сильнее или удобнее. Нет заявленного готового набора EH/Unwind findings, engineering score, confidence, HTML dashboard и Windows runtime collector.

Итог. Близкий программный аналог чтения данных, но не самостоятельный explainable detector.

Источники: [S25]

13.5. PE viewers и anomaly scanners

pestudio

Класс: зрелый статический PE malware triage продукт

Что подтверждено источниками. Официальные материалы описывают inspection PE без запуска, indicators, VirusTotal интеграцию, сведения о sections/imports/resources/certificates и другие malware-ориентированные признаки.

Где аналог сильнее. Шире охват PE и malware triage, готовый GUI, зрелая пользовательская база, больше общеизвестных indicators и внешних интеграций.

Где EH/Unwind Analyzer сильнее или удобнее. Проект глубже объясняет именно x64 unwind records, chains, cross-record integrity, reuse/dominance и выдаёт открыто документированную evidence matrix.

Итог. Сильный workflow-конкурент для первичного triage, но не подтверждён как столь же глубокий специализированный анализатор UNWIND_INFO.

Источники: [S9], [S10]

PE-bear

Класс: open-source cross-platform PE reversing tool

Что подтверждено источниками. Репозиторий позиционирует продукт как быстрый и гибкий инструмент исследования и изменения PE-файлов с визуальным интерфейсом.

Где аналог сильнее. Лучше интерактивный просмотр структуры и байтов, удобнее ручная навигация и визуальное исследование; может использоваться аналитиком ежедневно.

Где EH/Unwind Analyzer сильнее или удобнее. Проект лучше для batch/CI, воспроизводимых JSON reports и автоматического ранжирования EH/Unwind нарушений по явным правилам.

Итог. Не замена, а естественная парная связка: EH/Unwind Analyzer находит адрес и причину, PE-bear помогает вручную посмотреть файл.

Источники: [S8]

4n4lDetector

Класс: широкий open-source PE anomaly and malware triage tool

Что подтверждено источниками. Официальный репозиторий заявляет поиск PE anomalies, packer/anti-debug techniques, entropy, suspicious imports/sections и другие признаки.

Где аналог сильнее. Шире malware-oriented surface и больше типов общих PE сигналов; удобнее для общего первичного расследования подозрительного файла.

Где EH/Unwind Analyzer сильнее или удобнее. Проект имеет более узкую, формализованную и документированную модель x64 unwind, отдельные families/mitigations и статико-динамическую корреляцию.

Итог. Один из наиболее близких конкурентов по идее anomaly detector, но предметные модели разные: общий PE/malware triage против EH/Unwind integrity.

Источники: [S11]

13.6. Reverse-engineering платформы

IDA Pro / Hex-Rays

Класс: коммерческая профессиональная reverse-engineering platform

Что подтверждено источниками. Официальная документация описывает работу с exception handlers, функциями, disassembly/decompilation и широким набором процессоров и форматов.

Где аналог сильнее. Несопоставимо глубже анализ кода: disassembly, decompiler, CFG, types, xrefs, plugins, debugging и зрелый analyst workflow.

Где EH/Unwind Analyzer сильнее или удобнее. Проект дешевле по вычислительной и организационной сложности, полностью детерминирован для своей задачи, подходит для пакетного сканирования и сразу выдаёт формальный JSON/HTML с объяснением каждого сигнала.

Итог. IDA — не прямой detector-конкурент, а основная среда ручной верификации. Риск для проекта: аналогичный анализ можно реализовать IDAPython plugin.

Источники: [S12], [S13]

Ghidra

Класс: бесплатная open-source software reverse-engineering platform

Что подтверждено источниками. Официальный репозиторий описывает disassembly, decompilation, graphing, scripting и совместную работу с широким набором архитектур и форматов.

Где аналог сильнее. Глубже восстанавливает функции и контекст кода; имеет большую plugin/script экосистему; бесплатно решает значительную часть ручного RE.

Где EH/Unwind Analyzer сильнее или удобнее. Проект запускается как компактный CLI, выдаёт готовый explainable assessment и импортирует найденные функции/комментарии в Ghidra через собственный CSV importer.

Итог. Стратегически проект должен позиционироваться как preprocessor и evidence provider для Ghidra, а не как её замена.

Источники: [S14]

Binary Ninja

Класс: коммерческая RE platform с сильным API и intermediate languages

Что подтверждено источниками. Официальная документация описывает интерактивный анализ, API, automation, intermediate representations и debugger.

Где аналог сильнее. Сильнее по program analysis, data flow, scripting ergonomics, UI и возможности строить собственные анализаторы поверх IL.

Где EH/Unwind Analyzer сильнее или удобнее. Проект проще развернуть как независимый scanner, не требует коммерческой лицензии для просмотра отчёта и имеет предметно зафиксированные эвристики.

Итог. Конкурент за автоматизацию рабочего процесса, но также потенциальный потребитель JSON/API результатов проекта.

Источники: [S15]

radare2

Класс: open-source CLI/framework для reverse engineering

Что подтверждено источниками. Репозиторий заявляет disassembly, debugging, binary analysis, patching и scripting на множестве платформ.

Где аналог сильнее. Шире по форматам и архитектурам, сильнее как интерактивный CLI и программируемая платформа анализа кода.

Где EH/Unwind Analyzer сильнее или удобнее. Проект даёт более предметный и предсказуемый EH/Unwind report без необходимости собирать команды и scripts вручную.

Итог. Конкурирует за CLI-аудиторию. Главный риск — опытный пользователь может собрать часть проверок скриптами radare2, но это будет отдельная реализация.

Источники: [S16]

13.7. Debuggers и динамическая проверка

WinDbg

Класс: официальный Windows debugger

Что подтверждено источниками. Команда .fnent показывает function table entry и unwind information; debugger имеет прямой доступ к Windows runtime state, stack и exception events.

Где аналог сильнее. Глубже и точнее исследует конкретный процесс, нативно понимает Windows debugging, символы, memory, stack, exceptions и kernel/user contexts.

Где EH/Unwind Analyzer сильнее или удобнее. Проект автоматизирует массовый статический разбор до запуска, связывает его с ограниченным runtime collector и выпускает единый переносимый отчёт.

Итог. WinDbg сильнее для ручного доказательства конкретного поведения. Проект сильнее как повторяемый triage pipeline. Они дополняют друг друга.

Источники: [S4]

x64dbg

Класс: open-source Windows user-mode debugger

Что подтверждено источниками. Официальный сайт описывает debugger для x64/x32 applications, scripting и plugins.

Где аналог сильнее. Значительно удобнее для интерактивного unpacking, stepping, memory inspection, breakpoints и plugins.

Где EH/Unwind Analyzer сильнее или удобнее. Проект автоматически расставляет предметные точки наблюдения по статически извлечённым handlers и function-table APIs и формирует schema-validated report.

Итог. x64dbg — ручная лаборатория, EH/Unwind Analyzer — узкий автоматизированный sensor. Для вредоносных файлов обоим нужна изолированная Windows VM.

Источники: [S17]

Ограничение собственной динамики

Dynamic collector проекта не заменяет debugger: он фиксирует только заранее определённые events, handler/API hits и ограниченный RIP trace. Он не выполняет полное instruction tracing, symbolic execution, unpacking, taint/data-flow analysis или восстановление всех runtime-generated function tables.

Практическая конкурентная связка

Наиболее реалистичный рабочий процесс: проект выполняет быстрый статический triage → аналитик открывает representative RVA в Ghidra/IDA/PE-bear → при необходимости подтверждает поведение в WinDbg/x64dbg → JSON сохраняется в БД и связывает артефакты.

13.8. Смежные security-инструменты

Microsoft BinSkim

Класс: static binary security scanner

Что подтверждено источниками. Репозиторий описывает проверку security properties и compiler/linker settings, масштабируемый анализ и SARIF output.

Где аналог сильнее. Зрелая security engineering модель, стандартный SARIF, применение в build pipelines и проверка защитных свойств binaries.

Где EH/Unwind Analyzer сильнее или удобнее. Проект глубже именно в x64 unwind integrity и runtime evidence; находит те классы метаданных, которые не являются основной задачей BinSkim.

Итог. Смежный CI-конкурент. Полезное направление развития проекта — дополнительный SARIF export и совместный запуск в security pipeline.

Источники: [S18]

Mandiant capa

Класс: rule-based capability identification

Что подтверждено источниками. capa определяет возможности программы по features/rules и работает с executable code и результатами анализа.

Где аналог сильнее. Сильнее отвечает на вопрос «что код умеет делать», имеет развитую rule ecosystem и malware-analysis recognition.

Где EH/Unwind Analyzer сильнее или удобнее. Проект отвечает на другой вопрос: «корректны ли EH/Unwind structures и насколько необычно они организованы». Он работает даже до полноценного disassembly.

Итог. Не прямой конкурент. Совместный отчёт был бы сильнее: structural metadata evidence плюс semantic code capabilities.

Источники: [S19]

Detect It Easy

Класс: file type/compiler/packer identification

Что подтверждено источниками. Официальный репозиторий описывает signature и heuristic analysis для определения типов, packers, compilers и других file properties.

Где аналог сильнее. Шире и лучше для быстрого определения toolchain/packer и общего file triage.

Где EH/Unwind Analyzer сильнее или удобнее. Проект глубже проверяет внутреннюю согласованность exception/unwind metadata и формирует адресные findings для ручного анализа.

Итог. Смежный front-door tool. Результаты DiE могли бы использоваться как контекст, смягчающий или усиливающий отдельные эвристики проекта.

Источники: [S20]

pefile / readpe / PE Tree / XPEViewer

Класс: generic PE libraries and viewers

Что подтверждено источниками. Первичные проекты предоставляют чтение, отображение или редактирование PE structures; PE Tree использует pefile и интегрируется с IDA/Ghidra.

Где аналог сильнее. Шире базовый PE surface, удобнее как reusable library или visual inspector.

Где EH/Unwind Analyzer сильнее или удобнее. Проект содержит готовую предметную evidence model и dynamic correlation вместо простого представления полей.

Итог. Это инфраструктурные и workflow-аналоги, но не полный detector. Они снижают стоимость разработки конкурирующего parser frontend.

Источники: [S21], [S22], [S23], [S24]

13.9. В чём проект объективно сильнее

Слово «сильнее» здесь означает преимущество в конкретной узкой задаче, а не общую победу над более крупным продуктом.

- **Предметная специализация.** Проверяются не только отдельные поля, но и отношения между section map, exception directory, RUNTIME_FUNCTION, UNWIND_INFO, handler targets, chains, permissions, reuse и dominance.
- **Объяснимость.** Каждый finding имеет стабильный code, family, severity, weight, affected count, representative RVA/section и понятное объяснение.
- **Разделение подозрительности и уверенности.** Score не выдаётся за вероятность; confidence снижается mitigations при слабом, одиночном или маломасштабном сигнале.
- **Готовый analyst output.** Один запуск создаёт strict JSON, автономный HTML, CSV и Ghidra CSV; отчёт содержит SHA-256 и build provenance.
- **Static/dynamic bridge.** Статические handler/placement findings могут быть сопоставлены с handler hits, VEH/VCH/UEF и function-table API activity.
- **Кроссплатформенная безопасная статика.** Windows binaries можно разбирать на Linux/macOS без исполнения; динамика изолирована Windows-only модулем.
- **Малый deployment footprint.** Компактный C17 CLI легче встроить в backend, очередь задач или offline lab, чем полноценную RE platform.
- **Прозрачные правила.** Evidence matrix и пороги доступны для аудита, обсуждения на защите и последующей калибровки.
- **Robustness engineering.** Bounds checks, atomic reports, schema/semantic validation, sanitizer/mutation/fuzz gates уменьшают риск падений на malformed PE.
- **Privacy.** Анализ не требует загрузки файла в облако; HTML не использует сетевые assets.

Самое защищаемое преимущество
Главное отличие — не сам факт чтения UNWIND_INFO. Его умеют другие библиотеки и debuggers. Отличие состоит в объединении форматных инвариантов, cross-record эвристик, понятной evidence model, weakening rules и воспроизводимого отчёта.

13.10. В чём проект слабее конкурентов

• **Нет disassembly и decompilation.** Инструмент не сравнивает unwind codes с реальным прологом/эпилогом и не понимает семантику функции.

• **Узкая платформа.** Предметная модель ограничена PE32+ AMD64; нет ARM64, ARM64EC, x86 SEH, ELF/DWARF CFI, Mach-O compact unwind и kernel-specific models.

• **Нет UNWIND_INFO v3.** На 9 июня 2026 года parser принимает только версии 1 и 2; официальный Microsoft v3 будет помечен unsupported/parse failure.

• **Эвристики не статистически откалиброваны.** Пороги являются engineering defaults; нет ROC/precision/recall на большом репрезентативном corpus.

• **Недостаточно malicious/protected corpus evidence.** Benign compatibility подтверждена лучше, чем detection effectiveness на реальных protectors/malware.

• **Динамика ограничена.** Нет полного trace, emulation, symbolic execution, anti-anti-debug, DLL host orchestration и реконструкции всех runtime tables.

• **Нет полноценного GUI, project database и collaboration.** Крупные платформы лучше для длительной ручной работы нескольких аналитиков.

• **Нет plugin ecosystem.** IDA/Ghidra/Binary Ninja/radare2 расширяются сообществом и уже встроены в рабочие процессы.

• **Release ещё rc1.** Документация проекта сама требует независимый MSVC gate перед формальным тегом 1.0.0.

• **Низкий network effect.** Новый специализированный инструмент должен отдельно доказывать качество, поддерживать документацию и завоёвывать доверие.

Самое опасное слабое место на защите
Нельзя утверждать, что проект «правильно ловит все аномалии» или превосходит конкурентов по точности. Для такого вывода нужен размеченный comparative benchmark. Корректно утверждать: реализованные правила детерминированы, объяснимы и проверены на заявленных наборах; полнота обнаружения не доказана.

13.11. Критическое обновление стандарта: UNWIND_INFO v3

Факт

Microsoft Learn указывает, что начиная с Windows SDK 10.0.26100.0 определена версия 3 UNWIND_INFO. Страница обновлена 26 мая 2026 года. V3 вводит дополнительные возможности описания эпилогов через epilog descriptors и расширенный header layout. Это не гипотетическая будущая идея, а опубликованная текущая спецификация. [S2]

Что делает текущий parser

В src/unwind.c явно разрешены только **ver == 1 || ver == 2**. Любая другая версия получает UA_VERSION_UNKNOWN и завершает разбор как «Unsupported UNWIND_INFO version». Следовательно, корректный V3-файл способен получить parse failures и повышенный heuristic score.

Почему это важно для конкурентоспособности

- Новые MSVC/Windows SDK binaries со временем могут сделать V3 нормальным случаем.
- Без поддержки V3 появится систематический false positive, а не редкая edge case.
- LIEF и крупные RE platforms обновляются независимо; узкий инструмент должен оперативно следить за нормативной моделью.
- Документация и книга должны явно фиксировать, какую версию Microsoft spec поддерживает конкретный release.

Что требуется сделать

Приоритет	Работа
P0	Добавить V3 model/header parsing и отдельные unit fixtures по официальной спецификации.
P0	Не считать V3 повреждением, если структура корректна, даже до полной семантической поддержки.
P0	Добавить binaries, собранные актуальным SDK, в benign compatibility corpus.
P1	Сравнить декодирование V3 с MSVC/WinDbg/LIEF или другим независимым oracle.
P1	Обновить evidence matrix, docs, JSON semantics и defense materials.

Вывод для релиза

До реализации V3 проект можно выпускать только с явно указанной областью поддержки «UNWIND_INFO v1/v2». Для заявления «поддерживает современные Windows x64 unwind metadata» V3 должен стать release gate.

13.12. SWOT-анализ

Сильные стороны	Слабые стороны
Узкий экспертный фокус; объяснимые findings; cross-record checks; JSON/HTML/CSV; статико-динамическая корреляция; offline и cross-platform static.	Нет V3, disassembly, GUI/DB, статистической калибровки, широкого malicious corpus, множества архитектур и зрелой ecosystem.

Возможности	Угрозы
Plugin/API для Ghidra/IDA; SARIF; backend service; V3/ARM64EC; comparison with prolog; corpus/ROC; team reports; YARA/capa/DiE context.	Крупная платформа реализует такой же plugin; новые SDK меняют формат; heuristics дают false positives; недостаток corpus подрывает заявления о точности; narrow niche ограничивает рынок.

Защитимая конкурентная стратегия

• **Не конкурировать лоб в лоб с RE platforms.** Быть scanner/preprocessor, который сокращает время до первого содержательного адреса.

• **Сделать результат переносимым.** JSON Schema, SARIF, REST wrapper, stable finding codes и database-friendly metadata.

• **Усилить доказательность.** Публичный labeled corpus, differential tests, benchmark против LIEF/LLVM/WinDbg и статистическая калибровка.

• **Поддерживать стандарт.** V3 и новые architectures должны отслеживаться как release gates, а не случайные будущие улучшения.

• **Сохранить прозрачность.** Не превращать score в непрозрачную «AI-оценку» без проверяемой причины.

13.13. Приоритетный план усиления продукта

Приоритет	Улучшение	Конкурентный эффект
P0	UNWIND_INFO v3 + regression corpus	Убирает текущий нормативный пробел и false positives.
P0	Независимый MSVC release gate	Переводит rc1 в формально подтверждённый 1.0.0.
P0	Differential parser tests с LIEF/LLVM/WinDbg	Даёт внешний oracle и повышает доверие.
P1	Disassembly-проверка unwind codes против prolog/epilog	Закрывает главный разрыв с IDA/Ghidra.
P1	Размеченный benign/protector/malware corpus	Позволяет измерять precision/recall и калибровать thresholds.
P1	SARIF export и стабильный library/API facade	Упрощает CI, backend и security platform integration.
P1	DLL/PYD host dynamic runner и Unicode Win32 paths	Расширяет реальную динамическую применимость.
P2	Ghidra/IDA/Binary Ninja plugins	Встраивает findings в привычный analyst workflow.
P2	ARM64/ARM64EC и x86 SEH models	Расширяет рынок без смешивания архитектурных инвариантов.
P2	Web UI/БД как отдельный слой	Даёт collaboration, history, comparison и multi-sample analytics.

Что не следует делать первым

Красивый frontend не исправит parser gap, отсутствие benchmark и V3. Сначала нужны корректность, external oracles и corpus evidence; затем UI и коммерческая упаковка.

Минимальный критерий сильной версии 1.0

Поддержка актуальной Microsoft specification, чистый MSVC/MinGW/Linux/macOS gate, differential tests, воспроизводимый corpus report, документированные ограничения и стабильный JSON contract. Это важнее заявления «аналогов нет».

13.14. Вопросы комиссии: аналоги и конкуренция

В этом разделе: 36 вопросов. Отвечайте сравнительно: сначала класс инструмента, затем конкретное преимущество и обязательное ограничение.

Вопрос 183. Есть ли у проекта прямые аналоги?

Ответ. Полного публично документированного аналога всей комбинации функций в исследованном наборе не найдено. Ближайшие части реализуют LIEF и LLVM на уровне parsing, pestudio и 4n4lDetector на уровне PE triage, IDA/Ghidra на уровне ручного RE, WinDbg/x64dbg на уровне динамической проверки.

Осторожно. Не говорить «аналогов в мире нет»: непубличные продукты и плагины невозможно исключить.

Вопрос 184. Кто является главным конкурентом?

Ответ. Для parser/API — LIEF; для рабочего места реверс-инженера — IDA и Ghidra; для PE triage — pestudio и 4n4lDetector; для runtime verification — WinDbg и x64dbg. Главный конкурент зависит от того, какой слой системы сравнивается.

Вопрос 185. Почему нельзя просто использовать LIEF?

Ответ. LIEF уже предоставляет сильную объектную модель PE exceptions и может стать основой аналогичного продукта. Но готовые project-specific findings, evidence families, confidence mitigations, analyst reports и Windows runtime correlation поверх неё всё равно требуется реализовать.

Вопрос 186. Почему нельзя просто использовать llvm-readobj --unwind?

Ответ. llvm-readobj отображает unwind information как низкоуровневые данные. Проект дополнительно проверяет взаимные инварианты всего файла, агрегирует нарушения, ранжирует их и выдаёт машиночитаемый отчёт с шагами ручной проверки.

Вопрос 187. Чем проект лучше IDA или Ghidra?

Ответ. Он не лучше в общем reverse engineering. Он быстрее и формальнее решает одну узкую задачу: пакетно находит EH/Unwind anomalies и выдаёт объяснимый report. IDA/Ghidra намного сильнее для глубокого анализа кода и ручного подтверждения.

Осторожно. Нельзя называть проект заменой IDA/Ghidra.

Вопрос 188. Чем проект лучше PE-bear или pestudio?

Ответ. Он глубже специализируется на x64 RUNTIME_FUNCTION/UNWIND_INFO, chains, handlers, reuse, dominance и placement, а также документирует веса и ослабляющие правила. PE-bear удобнее для визуального просмотра, pestudio шире для общего malware triage.

13.14. Вопросы комиссии: продолжение

Вопрос 189. В чём уникальное торговое предложение?

Ответ. Explainable EH/Unwind triage: от bounds-safe parser до адресных findings, score/confidence, mitigations, JSON/HTML и статико-динамической correlation в одном компактном offline tool.

Вопрос 190. Можно ли доказать, что проект точнее конкурентов?

Ответ. Нет, текущих данных для такого утверждения недостаточно. Нужен одинаковый размеченный corpus, единые критерии, измерение precision/recall, false positives и coverage каждого инструмента.

Осторожно. Сравнение функций не является сравнением точности.

Вопрос 191. Почему отсутствие полного аналога не доказывает новизну?

Ответ. Потому что новизна требует анализа научных публикаций, патентов, продуктов и постановки задачи, а не только поиска похожего executable. Кроме того, всю функциональность можно теоретически воспроизвести plugin поверх зрелой RE platform.

Вопрос 192. Какой аналог наиболее опасен как будущий конкурент?

Ответ. LIEF плюс небольшой аналитический слой — самый реалистичный путь быстро повторить parser и часть эвристик. Ghidra/IDA/Binary Ninja plugins опасны тем, что сразу попадают в существующий workflow и используют готовый disassembly context.

Вопрос 193. Почему крупные платформы ещё не делают то же самое из коробки?

Ответ. EH/Unwind anomaly triage является узкой задачей относительно общего reverse engineering. Платформы обычно предоставляют parser, functions, scripts и UI, оставляя предметные политики detection плагинам и исследователям.

Вопрос 194. Можно ли встроить проект в Ghidra, IDA или Binary Ninja?

Ответ. Да. Уже существует консервативный Ghidra CSV importer. Следующий уровень — plugins, которые запускают core/CLI, проверяют JSON Schema и создают bookmarks/comments/types в адресном пространстве платформы.

13.14. Вопросы комиссии: продолжение

Вопрос 195. Зачем собственный parser, если есть pefile и LIEF?

Ответ. Собственный parser уменьшает dependency surface, позволяет точно контролировать bounds и diagnostics и демонстрирует предметную реализацию. Но для product quality полезны differential tests против независимых parsers.

Вопрос 196. Собственный parser — преимущество или риск?

Ответ. Оба. Преимущество — компактность и полный контроль. Риск — самостоятельная ответственность за каждый edge case и обновление specification. Поэтому нужны fuzzing, corpus tests и external oracles.

Вопрос 197. Что такое differential testing?

Ответ. Один и тот же corpus разбирается несколькими независимыми реализациями. Расхождения по ranges, unwind opcodes, handlers и parse status автоматически выделяются и исследуются. Это не доказывает правоту большинства, но быстро находит несовместимости.

Вопрос 198. Почему SARIF важен в сравнении с BinSkim?

Ответ. SARIF является стандартным форматом результатов static analysis и поддерживается security/developer tooling. JSON проекта богаче для предметной модели, но SARIF export упростит CI и агрегирование вместе с BinSkim и другими scanners.

Вопрос 199. Почему проект не использует машинное обучение?

Ответ. Текущая задача требует объяснимых структурных причин и работает при ограниченном размеченном corpus. Непрозрачная модель без качественных данных ухудшила бы доказательность. ML можно добавлять позже как дополнительный ranking layer.

Вопрос 200. Можно ли назвать score конкурентным преимуществом?

Ответ. Да, как прозрачную инженерную приоритизацию. Нет, если представлять его вероятностью атаки или статистически оптимальной оценкой: такой калибровки пока нет.

13.14. Вопросы комиссии: продолжение

Вопрос 201. Как новый UNWIND_INFO v3 влияет на проект?

Ответ. Это текущий compatibility gap. Parser поддерживает v1/v2 и считает v3 unsupported, поэтому правильные новые binaries могут получить parse failure и ложный высокий score. Поддержка v3 должна стать P0 release work.

Вопрос 202. Почему V3 не был учтён раньше?

Ответ. Сравниваемая реализация была зафиксирована до публикации обновлённой Microsoft документации от 26 мая 2026 года. Это нормальный пример того, почему форматный анализатор нуждается в постоянном specification monitoring.

Осторожно. Не отрицать проблему и не называть V3 несущественной.

Вопрос 203. Можно ли выпускать продукт без V3?

Ответ. Можно только с чёткой областью поддержки «UNWIND_INFO v1/v2» и известным ограничением. Для широкого заявления о современных Windows x64 binaries V3 должен быть реализован и проверен на реальном SDK corpus.

Вопрос 204. Как проверить реализацию V3?

Ответ. Нужны unit fixtures по спецификации, binaries актуального MSVC/SDK, negative mutations и differential comparison с независимым инструментом или WinDbg. Затем повторяются schema, semantic, sanitizer и corpus gates.

Вопрос 205. Какое главное преимущество проекта для заказчика?

Ответ. Он превращает трудоёмкий ручной осмотр EH/Unwind metadata в повторяемый первый проход: локализует адреса, объясняет причины, сохраняет результаты и передаёт их analyst/backend.

Вопрос 206. Какое главное ограничение для заказчика?

Ответ. Инструмент не определяет вредоносность и не заменяет reverse engineer. Для неизвестных toolchains, protectors и новых format versions findings требуют ручного подтверждения.

13.14. Вопросы комиссии: продолжение

Вопрос 207. Кто целевой пользователь?

Ответ. Reverse engineers, malware analysts, DFIR specialists, исследователи protectors, разработчики binary-analysis pipelines и команды, которым нужен batch/offline PE metadata triage.

Вопрос 208. Нужен ли проекту GUI для конкуренции?

Ответ. Для specialist CLI-аудитории не обязательно. Для массового product adoption нужны удобная навигация, history, comparison и collaboration, но их разумнее реализовать отдельным frontend/backend слоем поверх стабильного core JSON contract.

Вопрос 209. Что важнее: GUI или повышение точности?

Ответ. Сначала корректность parser, V3, differential tests и corpus-based validation. GUI улучшает удобство, но не исправляет false positives и coverage gaps.

Вопрос 210. Как защитить проект от копирования?

Ответ. Сами форматные правила не являются сильным барьером. Защита создаётся качеством corpus, регрессиями, интеграциями, стабильной schema, скоростью обновления specification и доверием пользователей.

Вопрос 211. Можно ли коммерциализировать такой узкий инструмент?

Ответ. Да как часть larger platform, enterprise scanner, forensic pipeline или expert toolkit. Отдельный массовый продукт имеет ограниченный рынок, поэтому особенно важны API, integrations и доказуемое сокращение analyst time.

Вопрос 212. Как измерить экономический эффект?

Ответ. На контролируемом наборе дел измерить время до обнаружения relevant RVA, долю файлов без ручного полного просмотра, число корректно приоритизированных случаев и время подготовки воспроизводимого отчёта.

13.14. Вопросы комиссии: продолжение

Вопрос 213. Какой benchmark нужен против конкурентов?

Ответ. Общий corpus из benign, malformed, protected и malicious PE; ground truth на уровне records; одинаковые категории сигналов; runtime scenarios; метрики parser agreement, precision/recall, crashes, throughput и analyst time.

Вопрос 214. Можно ли сравнить проект с антивирусом?

Ответ. Нет. Антивирус решает задачу обнаружения угроз на множестве signal layers. Проект исследует одну область metadata и выдаёт forensic/triage evidence, не malware verdict.

Вопрос 215. Можно ли сравнить проект с capa?

Ответ. Только как смежные уровни. capa определяет capabilities кода, а проект проверяет структуру EH/Unwind metadata. Их совместное использование даёт более полный контекст.

Вопрос 216. Как сформулировать конкурентное преимущество одним предложением?

Ответ. Это компактный explainable scanner, который превращает сложные Windows x64 EH/Unwind structures в адресные, проверяемые findings и переносимый отчёт, готовый для ручного RE и автоматизации.

Вопрос 217. Какой ответ будет слишком рекламным?

Ответ. «У проекта нет аналогов, он точнее IDA и обнаруживает скрытые атаки». Эти утверждения не подтверждены benchmark и смешивают разные классы инструментов.

Вопрос 218. Какой итоговый вывод о конкурентоспособности?

Ответ. Проект занимает реальную узкую нишу и имеет сильную explainability/integration model, но пока уступает зрелым платформам по breadth, code semantics, ecosystem и доказательности accuracy. После V3, differential tests и corpus benchmark его можно уверенно представлять как законченный specialist component.

13.15. Источники и дата проверки

Все ссылки проверены при подготовке исследования 9 июня 2026 года. Номер [S…] используется в тексте раздела. Для продуктов с plugin ecosystem отсутствие функции в документации не доказывает её отсутствия во всех сторонних расширениях.

[S1] Microsoft Learn — x64 exception handling
Нормативная модель RUNTIME_FUNCTION, UNWIND_INFO и раскрутки x64.
https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64

[S2] Microsoft Learn — x64 exception handling: UNWIND_INFO version 3
Опубликованное 26 мая 2026 года расширение UNWIND_INFO v3.
https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64?view=msvc-170#version-3

[S3] Microsoft Learn — PE format
Официальное описание PE/COFF, секций и exception directory.
https://learn.microsoft.com/en-us/windows/win32/debug/pe-format

[S4] Microsoft Learn — WinDbg .fnent
Штатный просмотр function table entry и unwind information.
https://learn.microsoft.com/en-us/windows-hardware/drivers/debuggercmds/-fnent--display-function-table-entry-

[S5] LIEF documentation — PE exceptions
API-модель исключений PE и RuntimeFunctionX64.
https://lief.re/doc/latest/formats/pe/exceptions.html

[S6] LIEF 0.17.0 changelog
Добавленная расширенная поддержка PE exception information.
https://lief.re/doc/stable/changelog.html#version-0-17-0

[S7] LLVM Command Guide — llvm-readobj
Опция --unwind и машинно-ориентированный просмотр объектных структур.
https://llvm.org/docs/CommandGuide/llvm-readobj.html

[S8] PE-bear official repository
Кроссплатформенный визуальный инструмент исследования PE.
https://github.com/hasherezade/pe-bear

[S9] pestudio official product page
Широкая статическая PE-malware triage без выполнения файла.
https://www.winitor.com/tools/pestudio/current/pestudio.html

[S10] pestudio features
Заявленные возможности, indicators и интеграции продукта.
https://www.winitor.com/tools/pestudio/current/pestudio-features.pdf

[S11] 4n4lDetector official repository
Широкий PE-анализатор аномалий, packer/anti-debug и подозрительных свойств.
https://github.com/4n4lDetector/4n4lDetector

[S12] Hex-Rays — exception handlers
Работа IDA с exception-handler metadata и границами функций.
https://docs.hex-rays.com/user-guide/user-interface/menu-bar/edit/functions

[S13] Hex-Rays IDA 9.2 release notes
Текущее развитие широкого коммерческого reverse-engineering комплекса.
https://docs.hex-rays.com/release-notes/9_2

[S14] Ghidra official repository
Полноценная платформа software reverse engineering с анализом кода.
https://github.com/NationalSecurityAgency/ghidra

13.15. Источники: продолжение

[S15] Binary Ninja documentation
Интерактивный анализ, API, промежуточные представления и автоматизация.
https://docs.binary.ninja/

[S16] radare2 official repository
Кроссплатформенный CLI/framework для disassembly, debugging и binary analysis.
https://github.com/radareorg/radare2

[S17] x64dbg official site
Windows user-mode debugger с plugin/script ecosystem.
https://x64dbg.com/

[S18] Microsoft BinSkim official repository
Static binary scanner для security/compiler settings и SARIF.
https://github.com/microsoft/binskim

[S19] Mandiant capa official repository
Rule-based определение возможностей исполняемого кода.
https://github.com/mandiant/capa

[S20] Detect It Easy official repository
Определение типов, packer/compiler и сигнатурно-эвристическое исследование файлов.
https://github.com/horsicq/Detect-It-Easy

[S21] pefile official repository
Python-библиотека чтения и исследования PE.
https://github.com/erocarrera/pefile

[S22] readpe official repository
Набор CLI-инструментов и библиотека для PE.
https://github.com/mentebinaria/readpe

[S23] PE Tree official repository
Визуализатор PE на базе pefile, доступный отдельно и в IDA/Ghidra.
https://github.com/blackberry/pe_tree

[S24] XPEViewer official repository
Кроссплатформенный viewer/editor PE.
https://github.com/horsicq/XPEViewer

[S25] Dissect documentation — PE exception directory
Программный доступ к AMD64 exception directory и unwind records.
https://docs.dissect.tools/en/stable/api/dissect/executable/pe/exception.html

[S26] DFRWS — Stack Trace Analysis of Large Scale Android Applications
Исследовательский пример криминалистического использования unwind/stack metadata.
https://dfrws.org/presentation/stack-trace-analysis-of-large-scale-android-applications/

Финальная формула позиционирования

Не «замена профессиональным reverse-engineering платформам» и не «детектор вредоносности», а специализированный, объяснимый и автоматизируемый слой контроля Windows x64 EH/Unwind metadata, который сокращает ручной поиск и передаёт воспроизводимые evidence в Ghidra/IDA/debugger/backend workflow.

14. Актуальность, новизна и значимость проекта

Этот раздел нужен для защиты постановки работы. Он отделяет доказуемую прикладную новизну конкретной реализации от чрезмерных утверждений о мировой научной новизне.

Краткая формула актуальности
Windows x64 использует exception/unwind metadata как часть штатного механизма восстановления стека и обработки исключений. Эти structures одновременно полезны загрузчику, debugger и reverse engineer, но могут быть повреждены, нестандартно сформированы, изменены protector/runtime или использоваться как источник неоднозначности. Ручная проверка большого числа взаимосвязанных записей трудоёмка; специализированная автоматизация сокращает время анализа и делает выводы воспроизводимыми.

Почему тема актуальна технически

• **Metadata участвует в выполнении системы.** RUNTIME_FUNCTION и UNWIND_INFO нужны Windows для table-based exception handling и virtual unwind nonleaf functions; это не декоративные поля PE. [S1], [S3]

• **Metadata является источником восстановленных функций.** Диапазоны, stack actions и handler addresses полезны для debugging, crash analysis, forensics и reverse engineering.

• **Ошибки взаимосвязаны.** Отдельно корректное поле ещё не гарантирует корректность section mapping, range ordering, chain target, handler placement или общей таблицы.

• **Объём превышает удобство ручной проверки.** Реальные binaries содержат сотни и тысячи runtime-function records; проверка overlaps, reuse и dominance требует агрегации всего image.

• **Формат развивается.** Появление официальной UNWIND_INFO v3 в актуальном SDK показывает, что проблема совместимости не застыла и требует постоянного specification monitoring. [S2]

• **Динамические tables дополняют файл.** JIT и runtime code могут регистрировать function tables через Windows API, поэтому только статическое представление не всегда описывает фактическое состояние процесса.

Почему тема актуальна для reverse engineer

• Повреждённые ranges и chains затрудняют корректное восстановление границ функций.

• Handler targets и registrations помогают найти управляющую логику runtime, protector или exception-driven control flow.

• Необычное placement/reuse само по себе не доказывает атаку, но задаёт конкретные адреса и гипотезы для ручного исследования.

• Структурированный report позволяет повторить вывод другому аналитику и связать его с исходным SHA-256, версией инструмента и конкретным finding.

14.1. Виды новизны: что можно утверждать

Уровень	Корректная формулировка	Что нельзя утверждать без доказательств
Научная	Предложена формализованная интерпретируемая модель совокупной оценки EH/Unwind anomalies с разделением suspiciousness, confidence, evidence families и weakening rules.	«Открыт новый закон», «впервые в мире найден способ разбирать UNWIND_INFO».
Методическая	В одном workflow объединены formal invariants, contextual heuristics, cross-record combinations, manual validation policy и static/dynamic correlation.	«До проекта никто не проверял exception metadata».
Программно-техническая	Создан самостоятельный bounds-safe C17 analyzer с reproducible reports, schema, provenance, Ghidra bridge, Windows collector и release tests.	«У проекта нет ни одного аналога» или «он лучше всех RE tools».
Прикладная	Сложные low-level structures преобразуются в адресные объяснимые findings, которые сокращают ручной triage и пригодны для backend/БД.	«По score автоматически устанавливается вредоносность».

Наиболее безопасная формула новизны для ВКР

Новизна работы состоит не в изобретении формата PE или алгоритма virtual unwind, а в разработке и реализации целостной объяснимой методики контроля Windows x64 EH/Unwind metadata: от безопасного разбора и cross-record checks до evidence model, ослабления неоднозначных эвристик, воспроизводимых отчётов и ограниченной статико-динамической корреляции.

Из каких конкретных элементов складывается новизна

- Пять evidence families вместо плоского списка предупреждений.
- Раздельные score, severity, confidence, evidence level и case strength.
- Mitigations, явно понижающие уверенность при одиночном, малом или placement-only сигнале.
- Combined findings для взаимного подтверждения placement, integrity, reuse и handler evidence.
- Переход от агрегата к representative RVA/section и manual validation steps.
- Один report contract для static structures и Windows runtime observations.
- Консервативный Ghidra importer, который не разрушает существующие analyst names и overlapping functions.

14.2. Теоретическая и практическая значимость

Теоретическая и методическая значимость

- Формализованы проверяемые invariants и отдельно выделены контекстные heuristics.
- Показано, почему suspiciousness нельзя отождествлять с вероятностью атаки.
- Предложена явная схема ослабления вывода при правдоподобном benign explanation.
- Определена traceable цепочка: input bytes → parsed model → finding → assessment → manual verification.
- Зафиксирована граница между format validation, anomaly detection и malware classification.

Практическая значимость

- **Для реверс-инженера:** сокращение пространства ручного поиска до конкретных ranges, UNWIND_INFO blobs, handlers и suspicious sections.
- **Для DFIR:** воспроизводимый offline report с SHA-256 и provenance, который можно приложить к исследованию.
- **Для разработчика:** диагностика malformed compiler/linker output, custom runtime tables и несовместимости metadata.
- **Для backend:** стабильный JSON Schema и deterministic codes позволяют сохранять, сравнивать и агрегировать результаты.
- **Для образования:** проект демонстрирует связь PE layout, ABI, exception handling, defensive parsing, heuristics и dynamic validation в одной системе.

Общественная и экономическая значимость

Проект потенциально снижает стоимость экспертного анализа Windows binaries: автоматизирует повторяющиеся проверки и сохраняет экспертное время для тех участков, где действительно требуется disassembly и контекст. Экономический эффект пока не измерен; его следует доказывать controlled study по analyst time, а не объявлять заранее.

Критерий полезности
Полезность проекта не зависит от того, классифицирует ли он malware. Даже обнаружение malformed metadata, локализация broken ranges и подготовка корректного Ghidra import имеют самостоятельную инженерную ценность.

14.3. Логика обоснования ВКР

Элемент	Формулировка применительно к проекту
Проблема	Ручной анализ большого массива связанных x64 EH/Unwind records трудоёмок и плохо воспроизводим.
Объект	PE32+ AMD64 images и их metadata обработки исключений/раскрутки стека.
Предмет	Структурные invariants, anomaly signs и методы static/dynamic проверки.
Цель	Создать инструмент безопасного анализа, локализации аномалий и формирования объяснимого отчёта.
Методы	Format parsing, invariant checking, heuristic aggregation, fuzz/testing, Windows debugging.
Результат	C17 core/CLI, reports, schema, evidence model, dynamic collector, Ghidra integration.
Новизна	Целостное explainable объединение указанных методов для узкой предметной области.
Практическая ценность	Сокращение triage и интеграция результатов в ручной и серверный workflow.
Ограничение	Не malware classifier; нет code semantics, V3 и статистически доказанной accuracy.

Причинная цепочка, которую нужно уметь произнести

От проблемы к результату

Метаданные необходимы Windows и полезны аналитику → записей много и они связаны → ручная проверка неполна и занимает время → generic viewers в основном показывают данные, а full RE platforms требуют ручной работы → поэтому нужен узкий automated triage → проект реализует parser, invariants, explainable heuristics и reports → аналитик получает проверяемые адреса и причины, но окончательный вывод делает с учётом disassembly и runtime context.

Критерии достижения цели

- Поддерживаемый PE разбирается без выхода за границы и с детальными diagnostics.
- Известные структурные нарушения дают ожидаемые findings.
- Benign corpus не получает необъяснимых structural findings в заявленной модели.
- Каждый отчёт проходит JSON Schema и semantic validation.
- Dynamic evidence не смешивается со static hypothesis и явно ограничено запуском.
- Результат можно передать другому специалисту и воспроизвести по hash/version.

14.4. Вопросы комиссии: актуальность и новизна

В этом разделе: 30 вопросов. Главный принцип ответа — доказывать конкретный вклад, не подменяя прикладную новизну недоказанным мировым первенством.

Вопрос 219. В чём актуальность темы?

Ответ. Windows x64 exception/unwind metadata является рабочей частью ABI и используется для восстановления стека. Она содержит множество взаимосвязанных записей, вручную проверять которые долго. Автоматическая локализация структурных противоречий и необычных patterns сокращает reverse-engineering triage и делает его воспроизводимым.

Вопрос 220. Почему тема актуальна именно сейчас?

Ответ. Windows x64 остаётся массовой платформой, JIT/protectors/custom runtimes используют exception infrastructure, а Microsoft продолжает развивать формат: в SDK 10.0.26100.0 появилась UNWIND_INFO v3. Это показывает живую, меняющуюся предметную область.

Вопрос 221. В чём научная новизна?

Ответ. Корректнее говорить о методической и прикладной новизне: разработана explainable model, которая объединяет formal invariants, contextual heuristics, evidence families, mitigations и static/dynamic correlation. Мировая научная новизна требует отдельного литературного и патентного обзора.

Осторожно. Не утверждать, что проект впервые научился читать UNWIND_INFO.

Вопрос 222. В чём программно-техническая новизна?

Ответ. В самостоятельной bounds-safe реализации единого workflow: PE/unwind parser, cross-record analysis, объяснимый assessment, versioned reports, Ghidra bridge, Windows runtime collector и проверяемые release gates.

Вопрос 223. В чём практическая значимость?

Ответ. Проект указывает конкретные ranges, blobs и handlers, которые нужно исследовать, сохраняет причины и provenance и может передавать JSON в backend/БД. Это уменьшает объём повторяющейся ручной работы.

Вопрос 224. В чём теоретическая значимость?

Ответ. Формализована граница между строгим нарушением, контекстной эвристикой и комбинированным evidence; отдельно определены suspiciousness, confidence и условия ослабления вывода.

14.4. Вопросы комиссии: продолжение

Вопрос 225. Почему обычного PE-viewer недостаточно?

Ответ. Viewer удобен для просмотра полей, но сам по себе не обязан проверять отношения всего набора records, вычислять ratios/reuse/dominance, применять mitigations и выпускать стабильный machine-readable assessment.

Вопрос 226. Почему не ограничиться скриптом?

Ответ. Скрипт может реализовать отдельную проверку, но законченный инструмент требует общей модели ошибок, безопасной обработки malformed input, reports, schema, tests, packaging и воспроизводимого поведения.

Вопрос 227. Какая проблема решается?

Ответ. Разрыв между низкоуровневыми raw structures и потребностью аналитика быстро понять, какие места противоречат формату или выглядят статистически необычно и почему их нужно проверить.

Вопрос 228. Как доказано, что проблема существует?

Ответ. Сама сложность модели следует из Microsoft PE/x64 exception specification: ranges, unwind codes, tails, handlers и chains взаимозависимы. Практическая трудоёмкость подтверждается объёмом runtime functions в реальном corpus и необходимостью ручных инструментов для проверки.

Вопрос 229. Почему это не искусственно узкая задача?

Ответ. Узкая задача оправдана высокой глубиной и самостоятельной ценностью metadata для OS, debugging и reverse engineering. Специализация позволяет сделать проверки объяснимыми и строгими, не смешивая их с общей malware classification.

Вопрос 230. Что является объектом и предметом?

Ответ. Объект — PE32+ AMD64 images и их exception/unwind metadata. Предмет — invariants, признаки повреждения или необычной организации и методы их статической и динамической проверки.

14.4. Вопросы комиссии: продолжение

Вопрос 231. Какова цель работы?

Ответ. Разработать инструмент, который безопасно разбирает Windows x64 EH/Unwind metadata, выявляет объяснимые anomalies, приоритизирует ручной анализ и выпускает воспроизводимый отчёт.

Вопрос 232. Какие задачи были решены для достижения цели?

Ответ. Изучена нормативная модель, реализованы PE/unwind parsers, evidence matrix, reports, dynamic collector, Ghidra importer, tests, fuzz/mutation validation и release packaging.

Вопрос 233. Как понять, что цель достигнута?

Ответ. Поддерживаемые inputs разбираются безопасно, test anomalies дают ожидаемые findings, reports валидируются, benign corpus обрабатывается совместимо, а результат содержит адреса и шаги ручной проверки.

Вопрос 234. В чём отличие результата от исходной идеи?

Ответ. Результат вышел за рамки parser prototype: появились evidence families, confidence и mitigations, JSON Schema, HTML/CSV, Ghidra integration, Windows runtime evidence, robustness tests и release workflow.

Вопрос 235. Что нового в системе признаков?

Ответ. Признаки разделены по происхождению и силе; hard structural defects не смешиваются с placement/reuse patterns; неоднозначные случаи ослабляются mitigations, а совпадающие семейства усиливаются combined findings.

Вопрос 236. Почему объяснимость важна?

Ответ. Reverse engineer должен проверить вывод в bytes, PE viewer, disassembler или debugger. Число без finding code, affected count, location и причины невозможно качественно защитить или воспроизвести.

14.4. Вопросы комиссии: продолжение

Вопрос 237. В чём значимость динамической части?

Ответ. Она позволяет отличить записанную в файле гипотезу от наблюдения конкретного запуска: увидеть handler hits, exception events, registrations и dynamic function-table API activity.

Вопрос 238. Почему динамика не доказывает всё поведение?

Ответ. Покрытие зависит от input, environment и времени запуска; anti-debug может изменить исполнение. Collector наблюдает ограниченные events, а не все возможные пути.

Вопрос 239. Какова значимость для заказчика?

Ответ. Заказчик получает repeatable component, который можно встроить в pipeline, хранить результаты по schema и сравнивать samples, не превращая low-level parser в ручную экспертную процедуру каждый раз.

Вопрос 240. Есть ли экономическая эффективность?

Ответ. Она ожидается за счёт сокращения analyst time, но ещё должна быть измерена. Корректный эксперимент сравнит время и качество triage с инструментом и без него на одном corpus.

Вопрос 241. Почему появление V3 подтверждает актуальность, а не провал проекта?

Ответ. Оно показывает, что стандарт развивается и нужен поддерживаемый analyzer. Одновременно это честно выявляет текущий gap реализации, который необходимо закрыть до широкого релиза.

Вопрос 242. Можно ли считать проект завершённой научной работой при наличии V3 gap?

Ответ. Да, если цель и область реализации явно ограничены v1/v2, результаты и ограничения зафиксированы, а V3 представлен как выявленное направление развития. Для заявления о полной современной product support gap нужно устранить.

14.4. Вопросы комиссии: продолжение

Вопрос 243. Как не преувеличить новизну?

Ответ. Не заявлять первенство без systematic literature/patent review, не называть parsing новым алгоритмом и не путать уникальную комбинацию engineering features с абсолютным отсутствием аналогов.

Вопрос 244. Какая самая сильная формулировка вклада автора?

Ответ. Автор формализовал предметную модель EH/Unwind evidence, реализовал её end-to-end в безопасном кроссплатформенном core, связал с Windows runtime observations и обеспечил проверяемость результата через tests и reports.

Вопрос 245. Что в работе можно повторить независимо?

Ответ. Сборку, regression/unit tests, mutation runs, schema/semantic validation, анализ corpus и проверку каждого finding по representative RVA и документированному правилу.

Вопрос 246. Почему воспроизводимость является вкладом?

Ответ. В binary analysis вывод часто зависит от версии инструмента и ручных действий. Hash, schema version, build provenance, deterministic rules и automated tests делают результат проверяемым другим исследователем.

Вопрос 247. Как проект связан с информационной безопасностью?

Ответ. Он поддерживает defensive reverse engineering и forensic triage: выявляет malformed или необычные служебные structures и помогает исследовать exception-driven mechanisms. Он не является средством эксплуатации.

Вопрос 248. Какой главный научно-практический результат?

Ответ. Не отдельный parser и не одна эвристика, а воспроизводимая цепочка от недоверенных bytes до ограниченного экспертного вывода с явными evidence, uncertainty и дальнейшими действиями аналитика.

15. Рыночная нужность и защита инженерных решений

Здесь проект рассматривается как потенциальный продукт и как инженерная система. Цель раздела — ответить на вопросы build-vs-buy, открытых компонентов, нейросетей, лицензирования, стоимости владения и реальной ценности для узкой аудитории.

Кому и зачем нужен именно такой продукт
Продукт нужен не массовому пользователю, а специалисту или backend pipeline, которому требуется быстро, одинаково и без облака проверить Windows x64 EH/Unwind metadata во множестве файлов, получить адресные причины и сохранить результат по стабильной schema. Полные RE platforms могут выполнить более глубокий анализ, но требуют больше ручной работы, лицензий, ресурсов и настройки.

Рыночная проблема

• Generic PE tools распыляют внимание на сотни свойств и не обязаны глубоко интерпретировать EH/Unwind consistency.

• Full RE platforms дороги по времени аналитика и избыточны для массового предварительного сканирования.

• Библиотеки вроде LIEF дают building blocks, но заказчику всё равно нужно разработать policy, reports, validation, deployment и поддержку.

• Облачные scanners не всегда допустимы для закрытых binaries, malware samples и материалов расследования.

• Результаты ручного просмотра плохо сравниваются между файлами и экспертами без стабильных finding codes и provenance.

Реалистичные продуктовые сегменты

Сегмент	Потребность	Форма поставки
Reverse engineering labs	Batch triage до Ghidra/IDA и адресные handoff notes.	CLI + HTML + plugins
DFIR / malware analysis	Offline repeatable evidence и связь с hash/case.	CLI + JSON + case DB
Software vendors	Проверка compiler/linker/runtime metadata regressions.	CI scanner + SARIF
Security platforms	Дополнительный structural signal в общей системе.	Library/service + schema
Education/research	Наблюдаемая связь ABI, PE, parser, heuristics и dynamics.	Source + corpus + book

15.1. Почему сделано собственное ядро

Короткий ответ

Не потому, что открытые проекты плохие, а потому что цель работы включает контролируемый defensive parser, точную предметную модель и доказуемую цепочку от bytes до finding. Сторонняя библиотека решила бы часть чтения формата, но не основную исследовательскую задачу.

Причины build, а не простой сборки из GitHub-компонентов

• **Контроль trust boundary.** Parser обрабатывает недоверенные bytes; владение bounds checks, integer arithmetic и diagnostics является частью качества.

• **Трассируемость решения.** Для каждого finding можно показать конкретный участок собственной модели и тест, а не непрозрачное поведение dependency.

• **Минимальная поверхность.** Узкий C17 core имеет меньше зависимостей, проще собирается offline и легче поставляется в изолированную лабораторию.

• **Стабильность контракта.** Внешняя API/ABI и representation стороннего parser могут меняться; project schema контролируется самим продуктом.

• **Образовательная и квалификационная цель.** ВКР должна демонстрировать понимание формата, defensive parsing и алгоритмов, а не только orchestration.

• **Предметные diagnostics.** Проект сохраняет точную parse_error и anomaly mask в форме, согласованной с heuristics и reports.

Почему всё же нужно использовать open source

• Как независимый oracle для differential testing, а не единственную истину.

• Для неключевых задач: JSON Schema validation, build system, test tooling.

• Как optional backend/adapter, если это уменьшит стоимость поддержки без потери контроля над evidence model.

• Для изучения edge cases и совместимости, соблюдая лицензии и attribution.

Честный недостаток собственного parser

Он повышает стоимость сопровождения и риск пропустить новый вариант формата, что уже видно на UNWIND_INFO v3. Поэтому правильная стратегия — сохранить собственную модель, но регулярно сравнивать её с LIEF/LLVM/WinDbg и актуальной Microsoft spec.

15.2. Почему не используется нейросеть

Фактор	Детерминированные правила	Нейросетевая модель
Ground truth	Формальные invariants выводятся из specification.	Нужен большой качественно размеченный corpus.
Объяснимость	Причина, адрес, threshold и weight видны.	Нужны дополнительные методы интерпретации.
Воспроизводимость	Одинаковый input и version дают одинаковый result.	Зависит от model/version/preprocessing.
Новые форматы	Нужно явно обновить parser/rules.	Без новых данных также возможен distribution shift.
False positives	Причина видна и может быть ослаблена mitigation.	Ошибку сложнее технически опровергнуть.
Deployment	Компактный offline C17 binary.	Weights/runtime/GPU или крупная CPU dependency.
Лучшее применение	Integrity validation и explainable triage.	Ranking слабых patterns при наличии большого corpus.

Почему ML сейчас методологически слабее

- Нет достаточно большого и репрезентативного labeled corpus EH/Unwind anomalies.
- Многие важнейшие проверки являются точными invariants; обучать модель тому, что EndAddress должен быть больше BeginAddress, бессмысленно.
- На защите и в forensic workflow нужно объяснить, какие bytes и правило вызвали вывод.
- Редкие toolchains и новые SDK создают distribution shift; модель может уверенно ошибаться без понятной причины.
- Score проекта не претендует на probability, поэтому правила не маскируются под псевдостатистическую модель.

Где ML может быть полезен позже

- Ранжирование weak/contextual patterns после обязательной formal validation.
- Поиск кластеров необычного reuse/handler behavior в большом corpus.
- Оценка similarity между toolchains/protectors при наличии labels.
- Предложение analyst priority, но не замена findings и raw evidence.

Правильная архитектура возможного ML

Rules остаются первым обязательным слоем и источником объяснимых features. ML добавляется отдельным необязательным ranking-компонентом с версией модели, calibration dataset и возможностью полностью отключить его.

15.3. Build-vs-buy, лицензии и supply chain

Вариант	Плюсы	Риски
Собственное ядро	Контроль, компактность, обучение, stable evidence model.	Стоимость поддержки, новые specs, ответственность за parser bugs.
LIEF/pefile/Dissect как core	Зрелый parsing surface, меньше собственного format code.	Dependency/API/license lifecycle; всё равно нужны heuristics/reports/dynamics.
Plugin для Ghidra/IDA	Готовый code context и analyst workflow.	Зависимость от platform/version/license; хуже standalone batch deployment.
Облачный сервис	Центральное обновление, collaboration, analytics.	Privacy, sample upload, network isolation и эксплуатационные затраты.

Почему нельзя просто скопировать код с GitHub

- Открытая публикация не означает отсутствие лицензии и условий использования.
- Нужно проверить license compatibility, notices, attribution и возможность коммерческого распространения.
- Чужой код увеличивает supply-chain surface: CVE, malicious update, abandoned dependency, transitive packages.
- Копирование без понимания не даёт доказательства correctness и не освобождает от собственных tests.
- Для ВКР необходимо явно отделять авторский вклад от reused components.

Как доказать самостоятельность проекта

- Показать architecture, commits/историю разработки, собственные models и tests.
- Привести список dependencies и лицензий.
- Сопоставить evidence matrix с конкретными functions исходного кода.
- Провести differential comparison: совпадение результатов не означает копирование, а расхождения становятся предметом анализа.

15.4. Рыночная жизнеспособность без рекламных обещаний

Почему узкий рынок не означает ненужный продукт

• В профессиональном security tooling ценность часто определяется стоимостью часа эксперта, а не числом массовых пользователей.

• Компонент может продаваться не отдельно, а как engine/API внутри larger platform.

• Узкая специализация облегчает доказательство качества и интеграцию в pipeline.

• Один предотвращённый длительный ручной разбор или обнаруженная compiler/runtime regression может оправдать использование инструмента.

Что должен доказать пилот

Метрика	Как измерять
Time-to-first-relevant-RVA	Одинаковые analysts и samples с инструментом и без него.
False-positive burden	Сколько findings требуют проверки и сколько получают benign explanation.
Parser coverage	Доля corpus, полностью декодированная без unsupported structures.
Analyst agreement	Насколько разные специалисты одинаково трактуют structured report.
Pipeline value	Скорость batch, размер отчёта, стабильность schema, стоимость хранения.
Detection utility	Ground-truth anomalies, precision/recall по категориям, не общий malware verdict.

Модель распространения

• Open-core или полностью open-source engine для доверия и исследований.

• Платный enterprise layer: orchestration, БД, access control, collaboration, support.

• Интеграционный SDK/API и plugins для существующих RE platforms.

• Экспертные услуги: corpus calibration, custom rules, закрытая лабораторная поставка.

Что является настоящим рыночным риском

Не отсутствие красивого frontend, а отсутствие измеренного преимущества по analyst time и отсутствие поддержки актуального формата. Рынок принимает узкий инструмент, если он надёжно экономит дорогую экспертную работу.

15.5. Враждебные вопросы комиссии и ответы

В этом разделе: 42 вопроса. Не спорьте с комиссией на уровне лозунгов. Признайте реальный риск, объясните инженерное решение и назовите способ независимой проверки.

Вопрос 249. Кому вообще нужен такой узкий продукт?

Ответ. Reverse-engineering, DFIR и malware-analysis teams, software vendors с custom runtimes и binary-analysis platforms. Их потребность — batch/offline проверка EH/Unwind metadata и быстрый переход к конкретным адресам, а не массовый пользовательский GUI.

Вопрос 250. Есть ли у проекта реальный рынок?

Ответ. Есть нишевой B2B/экспертный рынок, но его объём пока не измерен. Правильная следующая работа — interviews и pilot с метрикой analyst time. Без этого нельзя называть TAM или обещать коммерческий успех.

Вопрос 251. Почему заказчик не купит IDA и не решит всё?

Ответ. IDA нужна для глубокого ручного анализа, но не заменяет дешёвый batch scanner и database-friendly evidence contract. Проект может предварительно отбирать адреса и передавать их в IDA, экономя analyst time.

Вопрос 252. Почему не использовать бесплатную Ghidra?

Ответ. Ghidra сильнее анализирует код, но для одинакового automated EH/Unwind assessment всё равно нужен script/plugin, rules, reports и tests. Проект может быть таким внешним preprocessor и уже имеет Ghidra import.

Вопрос 253. Почему вы не собрали решение из LIEF и готовых библиотек?

Ответ. Потому что parser safety, exact diagnostics и предметная model входят в цель и авторский вклад. LIEF мог бы заменить часть parsing, но не evidence matrix, mitigations, reports, dynamic collector и validation. При этом LIEF полезен как independent oracle.

Вопрос 254. Разве писать свой parser не изобретение велосипеда?

Ответ. Было бы изобретением велосипеда без новой цели. Здесь собственный parser обеспечивает контроль trust boundary и интеграцию с уникальной evidence model. Риск дублирования компенсируется differential tests; если библиотека окажется надёжнее и дешевле, backend можно заменить без отказа от аналитического слоя.

15.5. Враждебные вопросы: продолжение

Вопрос 255. Почему вашему parser можно доверять больше, чем LIEF?

Ответ. Нельзя априори. Доверие строится на tests, fuzzing, corpus и сравнении с независимыми реализациями. Собственный parser даёт контроль и объяснимость, но не автоматическое превосходство.

Осторожно. Не заявлять, что собственный код по определению лучше зрелой библиотеки.

Вопрос 256. Вы копировали открытый код?

Ответ. Авторский вклад должен подтверждаться исходниками, architecture, tests и dependency/license inventory. Использование спецификаций и инструментов сравнения не является копированием. Любой reused code должен быть явно указан с лицензией и attribution.

Вопрос 257. Почему GitHub-код нельзя просто взять бесплатно?

Ответ. Публичный доступ не отменяет copyright и license terms. Кроме того, dependency нужно аудировать по безопасности, поддержке, API stability и transitive supply-chain risk.

Вопрос 258. Что если окажется, что аналогичный plugin уже есть?

Ответ. Это изменит утверждение об уникальности, но не уничтожит работу. Тогда нужно честно сравнить функции, correctness, reports и workflow. Самостоятельно реализованная и проверенная система остаётся инженерным результатом.

Вопрос 259. Почему не использована нейросеть?

Ответ. Для formal invariants нейросеть не нужна, а для слабых patterns нет достаточного размеченного corpus. Rules объяснимы, воспроизводимы и подходят forensic workflow. ML можно добавить позже только как дополнительный ranking layer.

Вопрос 260. Разве правила не примитивнее искусственного интеллекта?

Ответ. Простота не является недостатком, когда правило следует из спецификации. Нейросеть не делает проверку границ более правильной. Сложная модель оправдана только там, где она измеримо улучшает результат на независимом benchmark.

15.5. Враждебные вопросы: продолжение

Вопрос 261. Может ли нейросеть найти неизвестную аномалию?

Ответ. Потенциально да через outlier detection, но без ground truth она также будет выделять редкие нормальные toolchains. Такой сигнал полезен как гипотеза, а не доказательство; его нужно связывать с объяснимыми features.

Вопрос 262. Почему score не обучен на данных?

Ответ. Нет corpus достаточного качества для честной statistical calibration. Поэтому score прямо назван engineering triage scale, веса документированы, а проект не выдаёт его за вероятность.

Вопрос 263. Тогда веса взяты с потолка?

Ответ. Это экспертные engineering defaults, ранжирующие formal damage выше contextual patterns. Они не скрыты и могут быть проверены threshold sweep. Следующий этап — corpus-based calibration без изменения значения самих invariants.

Вопрос 264. Почему продукт нужен, если он не определяет malware?

Ответ. Reverse engineering включает восстановление структуры, диагностику malformed metadata и локализацию механизмов, а не только malware verdict. Инструмент экономит ручную работу и поставляет evidence другим detection layers.

Вопрос 265. Что покупает заказчик: parser или отчёт?

Ответ. Заказчик покупает повторяемый workflow: безопасное чтение, policy, приоритизацию, адресные доказательства, integration contract и поддержку формата. Parser является необходимой, но не достаточной частью.

Вопрос 266. Почему нельзя сделать всё одним Python-скриптом?

Ответ. Можно сделать prototype. Product требует контролируемой памяти, cross-platform binary, atomic reports, strict schema, tests, packaging и стабильного поведения на malformed inputs. Python adapter при этом может удобно оборачивать C core.

15.5. Враждебные вопросы: продолжение

Вопрос 267. Почему C, если он опасен для parser?

Ответ. C даёт переносимый компактный binary и прямой контроль bytes, но действительно повышает риск memory errors. Поэтому проект использует explicit bounds helpers, warnings-as-errors, ASan/UBSan, mutations и fuzzing.

Вопрос 268. Почему не Rust?

Ответ. Rust был бы сильным вариантом для memory safety. C выбран ради ABI, минимального runtime, Windows/Linux/macOS toolchains и учебной цели. Выбор не объявляется универсально лучшим; качество C реализации нужно доказывать тестами.

Вопрос 269. Почему нет frontend, если это продукт?

Ответ. Для specialist CLI и backend integration frontend не является обязательным. Core должен оставаться независимым, а GUI/БД строятся поверх JSON contract. Это снижает связность и не блокирует дальнейшую разработку.

Вопрос 270. Почему нет базы данных?

Ответ. БД отвечает за хранение, поиск и сравнение, а не за анализ bytes. Её правильнее добавить в backend layer, сохранив deterministic core и versioned JSON как границу.

Вопрос 271. Как продукт будет зарабатывать без GUI?

Ответ. Через enterprise engine/API, support, integration, private deployment и expert rules. GUI может повысить adoption, но monetization профессионального компонента не обязана зависеть от standalone desktop interface.

Вопрос 272. Что мешает крупному вендору скопировать проект?

Ответ. Форматные правила скопировать можно. Защитой являются corpus, regression history, интеграции, скорость поддержки новых specs и доверие. Поэтому moat нужно строить вокруг quality and workflow, а не секретности алгоритма.

15.5. Враждебные вопросы: продолжение

Вопрос 273. Почему проект не SaaS?

Ответ. Закрытые binaries и malware samples часто нельзя отправлять в облако. Offline-first engine является преимуществом. SaaS/on-prem orchestration можно добавить отдельно для организаций, где это допустимо.

Вопрос 274. Кто несёт ответственность за ложное срабатывание?

Ответ. Инструмент выдаёт triage finding и требует manual review, а не автоматический приговор. Ответственность продукта — корректно показать evidence и uncertainty; решение принимает уполномоченный аналитик.

Вопрос 275. Что будет при ложном отрицании?

Ответ. Файл может сохранить полностью корректную metadata или использовать неподдерживаемую технику. Поэтому нулевой score означает только отсутствие findings в модели и не исключает вредоносность.

Вопрос 276. Не слишком ли много ограничений для продукта?

Ответ. Ограничения неизбежны у любого analyzer. Сильный продукт не скрывает их, а делает machine-readable, тестируемыми и учитывает в workflow. Опаснее обещать универсальность, которой нет.

Вопрос 277. Почему комиссия должна считать работу самостоятельной?

Ответ. Потому что можно проследить авторскую предметную модель от требований и evidence matrix до C modules, tests, reports и dynamic correlation. Готовые tools используются для сравнения и проверки, а не заменяют основную реализацию.

Вопрос 278. Что является минимально продаваемой версией?

Ответ. Core с актуальным V3, подтверждёнными Windows gates, stable JSON, differential tests, documented deployment и пилотом, показывающим экономию analyst time. GUI может быть следующим слоем.

15.5. Враждебные вопросы: продолжение

Вопрос 279. Как доказать нужность рынку, а не словами?

Ответ. Провести 10–20 problem interviews, затем pilot на реальных задачах и измерить time-to-relevant-RVA, false-positive burden, repeatability и willingness to integrate или pay.

Вопрос 280. Что если интервью покажут, что отдельный продукт не нужен?

Ответ. Тогда ценность переносится в plugin, library или feature larger platform. Это не технический провал, а уточнение product form по данным рынка.

Вопрос 281. Почему рынок не решил задачу раньше?

Ответ. Части задачи решены parsers, viewers и RE platforms. Ниша остаётся в готовом explainable workflow именно для EH/Unwind anomalies. Её размер нужно подтвердить, а не объяснять отсутствием компетенции у рынка.

Вопрос 282. Какова стоимость владения?

Ответ. Низкая для standalone static CLI, но включает обновление specs, Windows validation, corpus maintenance и поддержку integrations. Именно эти постоянные работы определяют реальную стоимость, а не только размер executable.

Вопрос 283. Почему проект не поддерживает все архитектуры?

Ответ. Инварианты unwind architecture-specific. Попытка сразу охватить AMD64, ARM64, x86, DWARF и Mach-O ухудшила бы correctness. Правильное расширение — отдельные models за общим report contract.

Вопрос 284. Почему динамика Windows-only — это не архитектурный провал?

Ответ. Исследуемое runtime поведение реализуется Windows Debug API. Static core остаётся portable, а platform-specific collector изолирован модулем. Это сознательное разделение responsibilities.

15.5. Враждебные вопросы: продолжение

Вопрос 285. Можно ли использовать проект в CI?

Ответ. Да для static analysis: deterministic exit/report, JSON Schema и batch CLI подходят для pipeline. Для зрелой CI-интеграции полезно добавить policy exit modes и SARIF.

Вопрос 286. Что если third-party dependency найдёт аномалию иначе?

Ответ. Расхождение нужно сохранить как differential finding, проверить по Microsoft spec и реальному loader/debugger behavior. Ни одна реализация не объявляется безусловным oracle.

Вопрос 287. Почему продукт не использует VirusTotal?

Ответ. Это другой слой, требующий передачи hash/sample третьей стороне и не объясняющий EH/Unwind integrity. Интеграция может быть optional context, но core должен работать offline и без внешнего verdict.

Вопрос 288. Где граница авторского вклада?

Ответ. В предметной модели, parser/heuristics/report/dynamic integration, tests и release workflow. CMake, compiler, OS API и стандарты являются инфраструктурой; их использование не выдаётся за авторское изобретение.

Вопрос 289. Самый разрушительный вопрос: а что если проект никому не нужен?

Ответ. Это проверяемая product hypothesis, а не вопрос веры. Технический результат уже полезен как research/education component. Коммерческая форма должна быть подтверждена pilot; если standalone demand слаб, engine встраивается в существующие platforms.

Вопрос 290. Самый разрушительный вопрос: почему работа достойна защиты?

Ответ. Потому что она решает сформулированную low-level задачу end-to-end, содержит самостоятельную объяснимую модель, безопасную реализацию, static/dynamic components, формальные reports и воспроизводимую validation, при этом честно фиксирует границы и план закрытия V3/benchmark gaps.

16. Почему в модели именно 27 признаков

Главный ответ комиссии

Число 27 не выбиралось как целевой объём и не означает 27 возможных атак. Оно получилось после декомпозиции поддерживаемой модели на минимальные независимо объяснимые findings: базовые нарушения PE layout, целостность RUNTIME_FUNCTION, целостность UNWIND_INFO, статистические patterns, placement/permissions и четыре комбинированных подтверждения. Внутри aggregated finding unwind_anomalies дополнительно различаются 12 конкретных UA_* причин.

Что считается отдельным признаком

- У него есть отдельная причина и инженерное значение.
- Он может иметь собственную severity/weight и affected count.
- Аналитик может проверить его отдельным ручным действием.
- Он не является простой переформулировкой другого finding.
- Он стабильно сериализуется в report и может использоваться backend policy.

Почему не считать каждый UA_* отдельным итоговым finding

UA_VERSION_UNKNOWN, UA_TRUNCATED, UA_BAD_OPCODE и другие flags относятся к деталям одной декодируемой UNWIND_INFO record. На уровне сводного assessment они агрегируются как unwind_anomalies, чтобы score не увеличивался многократно из-за одного повреждённого blob. При ручном анализе конкретная UA_* причина всё равно сохраняется.

Почему число не должно быть больше ради впечатления

- Дублирующие признаки создают double counting и искусственно завышают score.
- Редкие неподтверждённые признаки увеличивают false positives.
- Каждый новый finding требует specification rationale, tests, documentation, report semantics и corpus calibration.
- Полезность определяется покрытием классов нарушений и explainability, а не количеством строк в списке.

16.1. Декомпозиция 27 findings по слоям

Слой	Кол-во	Findings	Что защищает
PE layout и вход в metadata	6	section raw/virtual OOB, raw/virtual overlap, misalignment, exception directory OOB	Корректность отображения file bytes в image и доступность exception table.
RUNTIME_FUNCTION table	7	no records, invalid ranges, RF OOB, unwind RVA OOB, unsorted, RF overlaps, duplicate Begin	Однозначность function ranges и ссылок на unwind records.
UNWIND_INFO integrity	2	parse fail, aggregated unwind anomalies	Безопасное декодирование header/codes/tail/chains; внутри 12 UA_* причин.
Reuse и handler statistics	3	moderate reuse, strong reuse, handler dominance	Выявление массовой централизации metadata/handlers.
Placement и permissions	5	unwind exec/unusual/writable, handler nonexec/writable	Соответствие роли данных и кода section map и memory permissions.
Комбинированные подтверждения	4	exec/nonexec handler, placement+integrity, reuse+dominance, writable pair	Усиление независимых, взаимно согласованных evidence без нового raw defect.
Итого	27	Пять evidence families и cross-family combinations.	Полная реализованная static assessment model.

27 findings не равны 27 низкоуровневым проверкам

Фактических проверяемых условий больше. Например, unwind_anomalies включает version, truncation, opcode, operand, CodeOffset, alignment, handler bounds и chain integrity. Итоговые findings — это устойчивый аналитический интерфейс, а не счётчик всех ветвей parser.

16.2. Какие вмешательства покрывает модель

Класс вмешательства	Какие признаки реагируют	Покрытие
Обрезка/повреждение файла или секции	raw OOB, virtual OOB, exception directory OOB, parse fail, UA_TRUNCATED	Прямое: структура становится физически недоступной.
Подмена размеров/адресов секций	OOB, overlaps, misalignment, unusual/exec/writable placement	Прямое для invalid layout, контекстное для необычного placement.
Изменение function Begin/End	invalid_ranges, rf_oob, unsorted, overlaps, duplicate_begin	Прямое при нарушении диапазонов; корректная согласованная подмена может не выявиться.
Перенаправление UnwindInfoRVA	unwind_rva_oob, parse_fail, unwind_anomalies, placement, reuse	Прямое при invalid target, эвристическое при корректном unusual target.
Порча unwind header/codes	parse_fail и 12 UA_* flags	Прямое для unsupported/truncated/invalid encoding в поддерживаемой модели.
Манипуляция handler RVA	UA_HANDLER_OOB, handler nonexec/writable, handler dominance, combinations	Прямое для OOB; контекстное для необычного, но допустимого адреса.
Манипуляция chained records	UA_CHAIN_INVALID, UA_CHAIN_LOOP, UA_CHAIN_DEPTH_EXCEEDED, parse fail	Прямое для invalid/cyclic/deep chain в пределах текущего лимита.
Массовое переиспользование metadata	reuse moderate/strong, reuse_dominance_combo	Статистический сигнал; legitimate compiler/runtime patterns возможны.
Централизация обработчиков	handler_dominance, placement signals, runtime handler hits	Контекстный сигнал; общий benign runtime handler возможен.
Writable/runtime-modifiable metadata	unwind_writable, handler_writable, writable combo, dynamic table APIs	Сильный context; JIT/runtime generation остаётся benign explanation.
Динамическая регистрация таблиц	RtlAddFunctionTable/RtlInstallFunctionTableCallback events	Покрывается runtime collector только на реально пройденном execution path.

16.3. Что 27 признаков не покрывают

• **Семантически ложная, но формально корректная metadata.** Если attacker согласованно подменил ranges и unwind codes, parser может не увидеть нарушения.

• **Несоответствие машинному прологу.** Без disassembly проект не доказывает, что unwind operations соответствуют фактическим инструкциям.

• **Невыполненные runtime paths.** Dynamic collector видит только события конкретного запуска.

• **Arbitrary memory writes.** Изменение таблиц без отслеживаемых APIs может остаться вне событий collector.

• **Не-EH/Unwind техники.** Imports, entropy, code injection, network behavior, capabilities и общая malware logic не входят в предмет модели.

• **Новые форматы.** UNWIND_INFO v3 и другие architectures требуют отдельной поддержки.

• **Законная нестандартность.** Custom linker, JIT, protector или runtime могут создать необычный, но корректный layout.

Почему это не опровергает модель

Модель заявлена как структурный anomaly detector и triage aid, а не полный detector любого вмешательства. Корректность оценивается относительно заявленной области: находит ли она реализованные structural contradictions и честно ли сообщает ограничения.

Как расширить покрытие

Пробел	Необходимое расширение
Metadata vs code	Disassembly и сопоставление unwind operations с prolog/epilog.
Runtime-generated tables	Снимок зарегистрированных tables и разбор их records после API call.
Arbitrary modifications	Memory watchpoints, ETW/instrumentation или hypervisor-assisted tracing.
Новые standards	V3, ARM64/ARM64EC, x86 SEH как отдельные architecture models.
Detection effectiveness	Размеченный interference corpus и per-category benchmark.

16.4. Как доказывать достаточность 27 признаков

Достаточность нельзя доказать одной фразой «признаков много». Нужна traceability matrix: specification invariant или intervention class → check → finding → test fixture → expected report → ручной oracle.

Уровень доказательства	Что должно существовать
Specification coverage	Список поддерживаемых invariants Microsoft PE/x64 unwind и ссылка на код.
Finding coverage	Для каждого из 27: positive fixture, benign negative case и report assertion.
Internal anomaly coverage	Для каждого UA_*: точечная mutation и ожидаемый bit/parse status.
Intervention coverage	Набор controlled mutations по адресу, размеру, opcode, handler, chain и permissions.
Corpus compatibility	Benign binaries разных toolchains/SDK, включая V3 после реализации.
Differential evidence	Сравнение decoded model с LIEF/LLVM/WinDbg на одинаковых samples.
Statistical calibration	Размеченный corpus и метрики по weak/contextual heuristics.

Когда можно сказать «покрывают вмешательства»

Только в ограниченной формулировке: набор покрывает реализованные классы структурных вмешательств и unusual organization, перечисленные в traceability matrix. Нельзя утверждать, что он покрывает любое вмешательство в EH/Unwind, пока нет сопоставления metadata с code, полного runtime monitoring и актуальной поддержки всех версий формата.

Сильный ответ на «не мало ли 27?»

Количество само по себе не является метрикой покрытия. 27 — это 27 независимых итоговых классов evidence; под ними находятся десятки низкоуровневых условий. Достаточность оценивается матрицей требований и mutation tests. Если новый независимый класс вмешательства не отображается ни в один finding, тогда нужен 28-й; добавлять дубликаты ради числа неправильно.

16.5. Вопросы комиссии по системе признаков

В этом разделе: 30 вопросов. Защищайте не магическое число 27, а метод построения, traceability, coverage boundaries и отсутствие double counting.

Вопрос 291. Почему признаков именно 27?

Ответ. Это результат декомпозиции поддерживаемой модели на независимо объяснимые findings: 6 PE layout, 7 RUNTIME_FUNCTION, 2 UNWIND integrity, 3 statistical, 5 placement и 4 combined. Число не задавалось заранее.

Вопрос 292. Не слишком ли мало 27 признаков?

Ответ. Количество не показывает coverage. Под 27 итоговыми findings находятся дополнительные низкоуровневые checks и 12 UA_* anomalies. Нужна traceability matrix по классам вмешательств, а не гонка за большим числом.

Вопрос 293. Не слишком ли много 27 признаков?

Ответ. Нет, если каждый независим, тестируем и ведёт к отдельному analyst action. Дублирующие условия объединены, а внутренние UA_* не начисляют отдельный score, чтобы избежать double counting.

Вопрос 294. Кто определил, что 27 достаточно?

Ответ. Это текущая engineering model, а не универсальный стандарт. Её достаточность проверяется относительно заявленных invariants и intervention classes. Новое независимое нарушение должно приводить к расширению модели.

Вопрос 295. Покрывают ли они любое вмешательство в EH/Unwind?

Ответ. Нет. Они покрывают перечисленные structural contradictions, placement и statistical patterns. Формально корректная согласованная подмена и несоответствие реальному коду могут остаться незамеченными.

Вопрос 296. Какие вмешательства покрываются лучше всего?

Ответ. Повреждение bounds, ranges, overlaps, RVA mapping, malformed unwind encoding, invalid handlers/chains и противоречивые permissions. Это прямые признаки, близкие к specification invariants.

16.5. Вопросы по признакам: продолжение

Вопрос 297. Какие вмешательства покрываются только эвристически?

Ответ. Unusual section names, writable/executable placement, массовое reuse и handler dominance. Они могут быть законными, поэтому имеют context, thresholds и mitigations.

Вопрос 298. Почему нет отдельного finding для каждого opcode?

Ответ. Opcode-level reasons хранятся во внутренней UA_* mask. Итоговый finding агрегирует повреждение записи, чтобы одна record не завысила score несколькими производными симптомами.

Вопрос 299. Почему combined findings не являются дубликатами?

Ответ. Они не повторяют raw condition, а фиксируют взаимное подтверждение двух evidence. Их вес ниже базовых сильных нарушений и отражает снижение вероятности случайного benign объяснения.

Вопрос 300. Почему признаки разделены на families?

Ответ. Несколько сигналов одной природы менее убедительны, чем независимые подтверждения из table integrity, unwind integrity, placement, reuse и handler behavior. Families нужны для confidence и case interpretation.

Вопрос 301. Как избежать двойного учёта одного повреждения?

Ответ. Internal anomalies агрегируются, affected_count не превращается в линейный score, related findings имеют ограниченные weights, а assessment разделяет score, families и confidence.

Вопрос 302. Почему OOB получает больший вес, чем unusual section?

Ответ. OOB обычно нарушает возможность корректного отображения структуры в поддерживаемой модели. Необычное имя или placement может быть создано законным linker script и поэтому требует контекста.

16.5. Вопросы по признакам: продолжение

Вопрос 303. Почему проверяется сортировка таблицы?

Ответ. Упорядоченность обеспечивает ожидаемую организацию и эффективный поиск по BeginAddress. Инверсии могут указывать на malformed или нестандартную таблицу, но сами по себе слабее overlap/OOB.

Вопрос 304. Почему проверяется duplicate BeginAddress?

Ответ. Несколько descriptions для одного начала функции создают неоднозначность выбора EndAddress и UnwindInfoRVA. Это отдельный дефект, даже если ranges формально не выходят за image.

Вопрос 305. Почему reuse может быть нормальным?

Ответ. Compiler folding, runtime helpers или общие prologs могут законно переиспользовать UNWIND_INFO. Поэтому moderate reuse подавляется без другого сигнала, a strong threshold рассматривается как triage pattern.

Вопрос 306. Почему handler dominance может быть нормальным?

Ответ. Языковой runtime может направлять множество функций в общий personality/handler. Реализация требует достаточного количества handlers и учитывает location/context.

Вопрос 307. Как проверить coverage каждого finding?

Ответ. Для каждого code нужен controlled positive sample, benign negative sample и assertion по JSON fields. Для percentage thresholds дополнительно нужны boundary tests ниже, на границе и выше порога.

Вопрос 308. Что такое mutation coverage вмешательств?

Ответ. Исходный корректный PE изменяется одной контролируемой операцией: range, RVA, opcode, handler, chain или section flag. Ожидается конкретный finding без несвязанных срабатываний.

16.5. Вопросы по признакам: продолжение

Вопрос 309. Почему синтетические mutations недостаточны?

Ответ. Они дают точный ground truth, но не представляют всё разнообразие toolchains и protectors. Поэтому нужны реальные benign/protected/malicious samples и differential comparison.

Вопрос 310. Какой 28-й признак следует добавить?

Ответ. Не по числу, а после gap analysis. Наиболее важное расширение сейчас — V3 support и metadata-to-prolog consistency; последнее может породить отдельные mismatch findings после disassembly.

Вопрос 311. Может ли attacker обойти все 27?

Ответ. Да, сохранив формально корректную и статистически обычную metadata или переместив malicious logic за пределы исследуемой области. Это ограничение любой узкой anomaly model.

Вопрос 312. Тогда зачем нужны признаки, если их можно обойти?

Ответ. Обходимость не отменяет полезность: многие malformed, packed, experimental и corrupted cases оставляют структурные следы. Инструмент повышает стоимость сокрытия и ускоряет анализ обнаруживаемых случаев.

Вопрос 313. Покрывает ли модель JIT?

Ответ. Частично. Writable placement и dynamic function-table API activity создают context, но текущая версия не реконструирует полностью все runtime-generated records. Это направление расширения.

Вопрос 314. Покрывает ли модель protectors?

Ответ. Она покрывает те структурные и runtime patterns protector, которые попадают в текущую модель. Идентификация конкретного protector и универсальное unpacking не заявляются.

16.5. Вопросы по признакам: продолжение

Вопрос 315. Как связать 27 признаков с нормативными документами?

Ответ. Hard findings выводятся из PE/x64 exception invariants Microsoft. Contextual findings должны быть отмечены как engineering heuristics и обоснованы benign corpus и manual review policy.

Вопрос 316. Почему признаки не записаны как YARA rules?

Ответ. YARA хорошо сопоставляет byte/string patterns, но здесь нужны parsed ranges, graph-like chains, ratios, section permissions и cross-record aggregation. YARA может использовать результат, но не заменяет model.

Вопрос 317. Можно ли менять набор признаков после релиза?

Ответ. Да, но stable finding codes нельзя переиспользовать с новым смыслом. Новые rules требуют schema/version decision, tests, evidence documentation и анализа влияния на score.

Вопрос 318. Как сравнить систему признаков с конкурентом?

Ответ. Составить общую taxonomy intervention classes и проверить, какие классы каждый tool детектирует, как локализует, объясняет и сколько даёт false positives. Сравнивать только число indicators неправильно.

Вопрос 319. Какова самая сильная сторона 27 признаков?

Ответ. Их traceability: от raw invariant или pattern до finding code, location, score effect, mitigation и manual validation step.

Вопрос 320. Какова самая слабая сторона 27 признаков?

Ответ. Отсутствие code-semantic validation и статистической калибровки contextual thresholds на большом размеченном corpus.

16.6. Какова точность модели

Главный вывод

Единого доказанного процента «точности модели» сейчас нет. Можно отдельно назвать наблюдаемые результаты compatibility, regression detection и robustness, но нельзя вычислить универсальную accuracy, sensitivity, precision или F1 без достаточно большого независимо размеченного корпуса положительных и отрицательных случаев.

Какие показатели уже подтверждены

Проверка	Наблюдаемый результат	Что это доказывает	Чего не доказывает
Реальный benign corpus	106/106 файлов: valid reports, score 0; 174 795 RUNTIME_FUNCTION без parse failures/structural anomalies.	Совместимость и отсутствие false-positive findings на выбранном наборе.	Универсальную специфичность для всех Windows binaries.
Synthetic regression set	6/6 ожидаемых результатов: 2 benign и 4 целевых suspicious samples.	Известные встроенные сценарии и report semantics работают как задумано.	Sensitivity на реальных attacks/protectors.
Extended mutations	1 000/1 000 обработаны без аварии и некорректного отчёта.	Robustness parser/report pipeline на данном mutation strategy.	Правильную классификацию каждой мутации.
Windows dynamic validation	124/124 schema/semantic-valid reports; 4/4 focused smoke scenarios.	Работу instrumentation и report contract в проверенных сценариях.	Полноту runtime path coverage.

16.6. Точность: корректная статистическая интерпретация

Что можно посчитать на benign corpus

Если считать каждый из 106 файлов независимым benign case, наблюдаемая file-level specificity относительно правила «ненулевой finding/score = положительный сигнал» составила 106/106 = **100%**. При простой биномиальной модели нижняя граница двустороннего 95%-го Wilson interval составляет приблизительно **96,5%**. Однако эта граница описывает только выборку при предположении случайной и независимой репрезентативности, которой текущий corpus полностью не гарантирует.

Как правильно произнести процент

«На выбранном корпусе из 106 известных benign PE32+ AMD64 файлов наблюдаемая file-level специфичность составила 100%: ложных положительных срабатываний не было. Это результат конкретного корпуса, а не доказательство 100%-й специфичности на генеральной совокупности».

Почему нельзя посчитать общую accuracy

- Нет большого representative positive corpus с ground truth по каждому EH/Unwind intervention class.
- Synthetic suspicious samples сконструированы под известные rules и завышают оценку, если смешать их с реальными данными.
- Единицей ground truth может быть file, function record, finding или конкретное вмешательство; смешивать уровни нельзя.
- Некоторые contextual findings не являются бинарной ошибкой: unusual placement может быть истинным наблюдением, но иметь benign explanation.
- Нулевой score не означает отсутствие malware, а положительный finding не означает наличие malware; обычная malware-classification accuracy здесь неприменима.

Какие метрики пока неизвестны

Метрика	Текущий статус	Что нужно
Sensitivity / Recall	Не установлена.	Достаточно большой positive corpus по каждому intervention class.
Precision / PPV	Не установлена.	Независимая ручная разметка всех positive findings.
Overall accuracy	Не установлена.	Репрезентативный баланс positive/negative и фиксированный decision threshold.
F1	Не установлена.	Достоверные precision и recall.
ROC-AUC / PR-AUC	Не установлены.	Непрерывный score на большом labeled corpus и независимый test split.
Parser semantic accuracy	Частично проверена.	Differential oracle и byte/record-level ground truth.

16.6. План строгой оценки точности

Этап	Требование
1. Taxonomy	Зафиксировать классы вмешательств и единицу оценки: file, record и finding.
2. Corpus	Собрать benign, malformed, protector, JIT/runtime и malicious samples разных toolchains/SDK.
3. Ground truth	Двойная независимая разметка экспертами и разрешение разногласий.
4. Split	Разделить calibration/development и blind test sets по семействам, а не случайным копиям.
5. Baselines	Запустить LIEF/LLVM/PE tools и ручной oracle на тех же samples.
6. Metrics	Per-class recall, file/finding precision, specificity, F1, PR-AUC и bootstrap/Wilson intervals.
7. Ablation	Отключать families/mitigations и измерять вклад каждого решения.
8. Thresholds	Калибровать только на development set; test set использовать один раз.
9. Error analysis	Публиковать false positives/negatives и benign explanations, а не только средний процент.

Какой показатель должен быть главным

Для triage-системы важнее не одна overall accuracy, зависящая от доли benign файлов, а набор показателей: высокая specificity, чтобы не перегружать аналитика; per-class recall для формальных повреждений; precision на уровне findings; и время, сэкономленное до ручного подтверждения.

Почему нельзя назвать 100% точность

Числа 106/106, 6/6, 1 000/1 000 и 4/4 относятся к разным экспериментам и разным свойствам. Складывать их в единый процент методологически неверно. Корректный вывод: на выполненных проверках получены идеальные наблюдаемые результаты, но statistical detection accuracy всей модели ещё не установлена.

16.7. Вопросы комиссии о точности

В этом разделе: 12 вопросов. Не смешивайте correctness, compatibility, robustness и detection accuracy.

Вопрос 321. Какова точность модели в процентах?

Ответ. Универсальный процент пока не установлен. На выбранном benign corpus наблюдаемая file-level specificity составила 106/106 = 100%, а на шести synthetic regression samples получено 6/6 ожидаемых результатов. Эти результаты нельзя объединять в общую accuracy.

Вопрос 322. Почему нельзя сказать, что точность 100%?

Ответ. Потому что выборки ограничены и не содержат репрезентативного набора реальных положительных случаев. Ноль ошибок на текущем corpus не гарантирует ноль ошибок на других toolchains, protectors и format versions.

Вопрос 323. Что означает 106/106 benign?

Ответ. Все 106 выбранных известных benign файлов дали valid report и нулевой score без структурных unwind anomalies. Это наблюдаемая совместимость и отсутствие false positives на конкретном corpus.

Вопрос 324. Можно ли назвать specificity 100%?

Ответ. Можно только с уточнением «наблюдаемая на выбранном corpus». Для генеральной совокупности нужна репрезентативная случайная выборка; приблизительная нижняя Wilson граница для 106/106 равна 96,5%, но corpus bias остаётся.

Вопрос 325. Какова sensitivity?

Ответ. Не установлена, потому что нет достаточно большого независимо размеченного positive corpus по всем классам вмешательств. Четыре synthetic positive samples подтверждают регрессию, но не дают внешнюю sensitivity.

Вопрос 326. Какова precision?

Ответ. Не установлена. Нужны реальные predicted-positive cases и независимое экспертное подтверждение каждого finding. На одном benign corpus положительных предсказаний нет, поэтому PPV по нему не вычисляется.

16.7. Вопросы о точности: продолжение

Вопрос 327. Что доказывают 6/6 regression samples?

Ответ. Что два встроенных benign и четыре специально созданных suspicious сценария дают зафиксированные ожидаемые reports. Это functional regression, а не оценка генерализации.

Вопрос 328. Что доказывают 1 000 mutations?

Ответ. Устойчивость обработки в рамках mutation strategy: нет crash и некорректных reports. Это robustness, а не 1000 правильно классифицированных атак.

Вопрос 329. Что доказывают 124/124 Windows reports?

Ответ. Корректность schema/semantics и работоспособность dynamic pipeline в выполненных запусках. Это не измерение полноты наблюдения всех runtime paths.

Вопрос 330. Какая метрика важнее общей accuracy?

Ответ. Для triage важны per-class recall формальных дефектов, specificity и false-positive burden, precision findings и analyst time. Overall accuracy может быть искусственно высокой на преимущественно benign corpus.

Вопрос 331. Как получить научно обоснованную оценку?

Ответ. Создать taxonomy и blinded labeled corpus, отделить development от test set, провести двойную экспертную разметку и посчитать per-class metrics с confidence intervals и error analysis.

Вопрос 332. Как отвечать комиссии одной фразой?

Ответ. «На выбранном benign-корпусе из 106 файлов наблюдаемая специфичность равна 100%, но полная statistical accuracy модели пока не установлена из-за отсутствия большого независимо размеченного positive corpus».

17. Готовый сценарий защиты проекта

Ниже приведён логический каркас выступления. Его не нужно читать дословно: задача — сохранить причинную последовательность и не перескакивать к деталям раньше времени.

Слайд	Главная мысль	Что показать
1. Тема	Специализированный анализ PE32+ AMD64 EH/Unwind metadata.	Название и одно предложение.
2. Проблема	Ручная проверка связанных records трудоёмка и плохо воспроизводима.	RUNTIME_FUNCTION → UNWIND_INFO → handler.
3. Актуальность	Metadata нужна Windows и RE; format развивается; anomalies влияют на анализ.	Microsoft model и V3 как evidence развития.
4. Цель/задачи	От safe parser до explainable report и runtime evidence.	Краткая pipeline diagram.
5. Архитектура	Модульный монолит: core + CLI, Windows collector изолирован.	Modules/data flow.
6. Признаки	27 итоговых findings, 12 UA_* details, 5 evidence families.	Декомпозиция и priorities.
7. Оценка	Score — triage; confidence и mitigations ограничивают вывод.	Один пример combined evidence.
8. Динамика	Наблюдает handlers/API/events конкретного запуска.	Static/dynamic correlation.
9. Отчёты	JSON/HTML/CSV/Ghidra, hash и provenance.	Фрагмент отчёта.
10. Проверка	Unit/regression/schema/mutations/benign corpus/Windows smoke.	Только подтверждённые числа.
11. Аналоги	LIEF/PE tools/RE platforms решают части; проект — узкий explainable layer.	Competitor map.
12. Вклад	Целостная методика и реализация, а не новый формат или malware classifier.	Новизна и practical value.
13. Ограничения	V3, no disassembly, corpus calibration, limited dynamics.	Честный roadmap.
14. Итог	Сокращает путь от binary к проверяемой analyst hypothesis.	Один вывод без рекламы.

17.1. Текст выступления на 7–10 минут

Введение. Тема моей работы — разработка инструмента структурного анализа и выявления аномалий в Windows PE32+ AMD64 metadata обработки исключений и раскрутки стека. Эти данные представлены exception directory, RUNTIME_FUNCTION и UNWIND_INFO и используются операционной системой при восстановлении контекста nonleaf functions.

Проблема. В реальном binary таблица может содержать тысячи связанных records. Для корректной интерпретации недостаточно показать отдельные поля: необходимо проверить границы секций, отображение RVA, диапазоны функций, chains, unwind codes, handler addresses, permissions и статистические patterns. Ручной анализ занимает время и плохо воспроизводится.

Цель. Целью стало создание самостоятельного инструмента, который безопасно разбирает недоверенный PE-файл, локализует нарушения и необычные patterns, объясняет причину каждого вывода и формирует переносимый отчёт для дальнейшей работы реверс-инженера.

Архитектура. Проект реализован как модульный монолит на C17: отдельные modules отвечают за bytes и hashing, PE, unwind, heuristics, reports и Windows dynamic collector. Static core работает на Windows, Linux и macOS; dynamic collector изолирован и доступен только на Windows x64.

Система признаков. В модели 27 итоговых findings. Это не произвольное число: 6 относятся к PE layout, 7 к RUNTIME_FUNCTION table, 2 агрегируют UNWIND_INFO integrity, 3 описывают reuse и handler statistics, 5 — placement и permissions, 4 — взаимное подтверждение независимых сигналов. Внутри unwind_anomalies сохраняются 12 низкоуровневых UA_* причин.

Интерпретация. Формальные нарушения имеют больший приоритет, чем контекстные patterns. Score используется только для triage и не является вероятностью атаки. Confidence учитывает число независимых evidence families и снижается mitigations, если сигнал одиночный, sample мал или placement имеет нормальное объяснение.

Динамика. Windows collector запускает target под Debug API и фиксирует exceptions, handler hits, VEH/VCH/UEF registrations и обращения к APIs динамических function tables. Эти данные не заменяют debugger и отражают только конкретный execution path, но позволяют подтвердить часть статических гипотез.

Результат. Инструмент выпускает console, strict JSON, автономный HTML, CSV и Ghidra CSV; сохраняет SHA-256, version и build provenance. Ghidra importer переносит function ranges и warnings, не перезаписывая analyst names без необходимости.

17.1. Текст выступления: продолжение

Проверка. Реализация проверяется unit и regression tests, JSON Schema и semantic validators, strict compiler warnings, sanitizers, mutation/fuzz workflows, benign corpus и Windows dynamic smoke scenarios. Эти проверки подтверждают устойчивость и заявленную compatibility, но не доказывают универсальную detection accuracy.

Аналоги. LIEF и LLVM сильны как parsers/inspectors; pestudio, PE-bear и 4n4lDetector — как широкие PE tools; IDA, Ghidra, Binary Ninja и radare2 — как полноценные RE platforms; WinDbg и x64dbg — как debuggers. Проект не заменяет их. Его место — специализированный explainable preprocessor, который автоматизирует одну глубокую область и передаёт evidence этим системам.

Новизна и значимость. Новизна состоит не в изобретении PE или virtual unwind, а в целостной методике: safe parsing, cross-record checks, evidence families, mitigations, static/dynamic correlation и воспроизводимый report contract. Практическая ценность — сокращение пространства ручного поиска и возможность встроить результаты в backend или case database.

Ограничения. Проект не анализирует семантику всего кода, не определяет malware, не покрывает все execution paths и пока поддерживает UNWIND_INFO v1/v2. После публикации Microsoft specification v3 это стало приоритетным compatibility gap. Также необходимы differential tests и большой размеченный corpus для статистической калибровки contextual heuristics.

Заключение. Получен не просто parser, а законченная в заявленной области аналитическая цепочка от bytes до проверяемой гипотезы для reverse engineer. Сильная сторона проекта — объяснимость и воспроизводимость; дальнейшее развитие должно усиливать coverage и доказательность, не скрывая uncertainty.

Последняя фраза выступления
Таким образом, работа автоматизирует наиболее формализуемую часть анализа Windows x64 EH/Unwind metadata и превращает её в воспроизводимый источник адресных evidence, который дополняет, а не заменяет профессиональные средства reverse engineering.

17.2. Тактика ответа жёсткой комиссии

Тип атаки	Как отвечать
«Это уже есть»	Уточнить класс аналога, признать пересечение, затем назвать отличия workflow и evidence model.
«Это никому не нужно»	Отделить техническую полезность от product hypothesis и предложить измеримый pilot.
«Мало данных»	Согласиться с границей, не делать statistical claims, назвать план corpus/benchmark.
«Почему не AI?»	Показать formal invariants, отсутствие labels и требования explainability; ML оставить ranking layer.
«Почему свой parser?»	Обосновать trust boundary и вклад, затем признать maintenance risk и differential testing.
«27 мало/много»	Перевести разговор с числа на taxonomy, traceability и mutation coverage.
«Нашли V3, значит проект сломан»	Признать compatibility gap, ограничить scope v1/v2 и назвать P0 validation plan.
«Где точность?»	Не подменять correctness parser точностью detector; запросить/описать общий labeled benchmark.
«Это malware detector?»	Нет; это structural anomaly detector и analyst triage component.
«Проект завершён?»	В заявленной v1/v2 области core реализован; для широкого 1.0 нужны V3, MSVC gate и benchmark.

Формула сильного ответа

Факт → граница → решение → проверка

1. Назвать подтверждённый факт. 2. Признать точную границу. 3. Объяснить, почему выбрано текущее решение. 4. Назвать тест или эксперимент, который может проверить ответ независимо. Такая структура сильнее попытки любой ценой доказать, что проект идеален.

Фразы, которые усиливают защиту

- «В пределах заявленной модели и версии формата…»
- «Это формальное нарушение; в отличие от него следующий сигнал является контекстной эвристикой…»
- «Текущих данных недостаточно для статистического утверждения, поэтому…»
- «Этот вывод можно независимо проверить следующим способом…»
- «Конкурент сильнее в анализе кода, а наш инструмент — в пакетной формализации именно этой области…»

Фразы, которые разрушают защиту

- «Аналогов вообще нет».
- «27 признаков покрывают любые атаки».
- «Score показывает вероятность вредоносности».
- «Нейросеть не нужна, потому что правила всегда лучше».
- «Собственный parser точно надёжнее LIEF».
- «Проект полностью закончен и больше ничего делать не нужно».

17.3. Финальная памятка полной защиты

Нужно помнить	Точная позиция
Что это	Explainable structural anomaly analyzer для PE32+ AMD64 EH/Unwind metadata.
Что не это	Не antivirus, не universal malware classifier, не замена IDA/Ghidra/debugger.
27 признаков	27 итоговых evidence classes; под ними 12 UA_* и множество low-level checks.
Актуальность	Рабочая Windows ABI metadata, большие таблицы, сложные связи, живой evolving format.
Новизна	Интегрированная methodology и implementation, не изобретение PE/unwind.
Рынок	Нишевой professional component; ценность должна подтверждаться analyst-time pilot.
Почему своё ядро	Trust boundary, traceability, minimal deployment; риск закрывается external oracles.
Почему без AI	Formal invariants + нет labels + explainability; ML возможен как optional ranking.
Главная сила	Адресные, проверяемые, воспроизводимые evidence и stable report contract.
Главная слабость	Нет code semantics/V3/statistical calibration и полного runtime coverage.
Главный следующий шаг	V3, differential tests, corpus benchmark, затем SARIF/plugins/UI.
Стиль ответа	Точный факт, честная граница, инженерное решение, независимая проверка.

Главная мысль всей защиты

Сила работы не в обещании обнаружить всё, а в том, что для ограниченной и технически важной области построена полная, объяснимая и проверяемая цепочка анализа. Комиссии нужно показать понимание не только возможностей, но и границ, потому что именно это отличает инженерный продукт от демонстрационного прототипа.

18. Сценарий демонстрации проекта

Демонстрация должна доказывать три вещи: инструмент запускается, различает чистый и контролируемо повреждённый sample и позволяет перейти от finding к конкретному месту ручной проверки.

Шаг	Действие	Что произнести
1	Показать --version.	Версия tool/schema/build фиксируется в каждом report.
2	Запустить benign sample с --heuristics --json --html.	Ожидается score 0 и отсутствие findings в текущей модели.
3	Открыть JSON metadata.	Показать SHA-256, schema, sections, functions и assessment.
4	Запустить suspicious_unwind_oob.	Показать unwind_rva_oob/parse_fail и representative location.
5	Сравнить score и confidence.	Score ранжирует, confidence описывает силу интерпретации.
6	Открыть HTML.	Показать автономный analyst-friendly report без внешних assets.
7	Сгенерировать Ghidra CSV.	Результат переносится в ручной RE workflow.
8	Показать dynamic report заранее.	Не запускать неизвестный binary на host; использовать проверенный VM artifact.
9	Показать тесты.	Сборка и CTest подтверждают regression/report contracts.

Правило безопасной демонстрации

На защите не запускать непроверенный интернет-sample на основном компьютере. Для динамики использовать изолированную Windows VM или заранее сохранённый schema-valid report с hash и описанием сценария.

Что подготовить заранее

- Два коротких sample: benign и один controlled suspicious.
- Команды в текстовом файле без длинных путей и случайных аргументов.
- Готовые JSON/HTML на случай медленной сборки.
- Скриншоты ключевых блоков отчёта для презентации.
- Проверенную Windows dynamic запись, а не live malware execution.

18.1. Резервный план при сбое демонстрации

Сбой	Действие	Как объяснить
Не собирается на чужом ПК	Показать release binary, package hash и CI/test evidence.	Сборка зависит от toolchain; reproducible artifact подготовлен заранее.
Нет Windows VM	Показать сохранённый dynamic JSON/HTML и validation report.	Dynamic mode platform-specific; evidence уже зафиксировано с provenance.
Команда долго выполняется	Перейти к заранее созданному report.	Анализ deterministic; live run не является единственным доказательством.
HTML не открывается	Показать JSON и console.	HTML — presentation layer, JSON — основной contract.
Sample удалён антивирусом	Использовать synthetic non-malicious fixtures.	Для доказательства mechanics malware не требуется.
Вопрос вне scope	Назвать границу и способ проверки.	Не импровизировать неизвестный факт.

Что нельзя делать при сбое

- Не менять код на глазах комиссии.
- Не скачивать случайный executable из интернета.
- Не объяснять любой сбой «антивирусом» или «Windows».
- Не выдавать заранее созданный report за только что выполненный запуск.
- Не спорить с фактом ошибки: отделить environment failure от core result.

Сильная фраза при сбое

«Live demonstration зависит от среды, поэтому я подготовил воспроизводимый artifact: в нём сохранены hash input, version инструмента, compiler/platform и schema-valid результат. Сейчас покажу тот же этап по отчёту».

19. Карта доказательств для ключевых утверждений

Утверждение	Доказательство	Ограничение
Static core кроссплатформенный	CMake presets/builds и отсутствие Windows API в static path.	Dynamic collector только Windows x64.
Parser устойчив к malformed input	Bounds helpers, sanitizers, mutations/fuzz, error paths.	Нельзя доказать отсутствие всех bugs.
Benign compatibility высокая	106/106 valid zero-score reports.	Corpus ограничен выбранными vendors/toolchains и v1/v2.
Известные anomalies обнаруживаются	6 regression fixtures и unit tests UA_*.	Synthetic set не измеряет external recall.
Reports воспроизводимы	SHA-256, version/schema/compiler/platform, validators.	Environment влияет на dynamic execution path.
Признаки объяснимы	Evidence matrix, stable codes, affected count, manual steps.	Thresholds ещё engineering defaults.
Динамика работает	124 reports, 4 focused smoke cases.	Нет полного path coverage и DLL host runner.
Проект имеет нишу	Gap между parser/viewer/full RE platform и specialized report.	Market demand требует interviews/pilot.
Новизна существует	Уникальная интеграция methods в одной реализации.	Мировое первенство не доказано.
Модель точна	Только наблюдаемые corpus/regression результаты.	Общая statistical accuracy неизвестна.

Красные линии доказательности

- Не превращать отсутствие findings в доказательство безопасности.
- Не превращать handler hit в доказательство malicious intent.
- Не превращать 106 benign файлов в универсальную 100%-ю точность.
- Не превращать отсутствие найденного полного аналога в патентную новизну.
- Не превращать passing tests в доказательство отсутствия defects.

20. Чек-лист за 24 часа до защиты

Область	Проверить
Файлы	PDF защиты, презентация, release package, hashes, два samples, reports.
Команды	--version, benign analysis, suspicious analysis, report generation, tests.
Среда	Ноутбук/питание, VM snapshot, offline copies, шрифты, PDF reader.
Числа	Версия, tests, corpus, runtime functions, mutations, Windows reports.
Формулировки	Актуальность, новизна, 27 признаков, точность, конкуренты, ограничения.
Риски	V3, no disassembly, no calibrated positive corpus, limited dynamics.
Демо	Проверить с нуля по таймеру; иметь screenshots и готовые reports.
Безопасность	Не запускать untrusted sample вне изолированной VM.

Пять чисел, которые нужно знать точно

- **27** итоговых findings.
- **12** внутренних UA_* anomaly reasons.
- **106/106** benign corpus reports без ненулевого score.
- **174 795** разобранных RUNTIME_FUNCTION в этом corpus.
- **1 000** расширенных mutations без crash/invalid report.

Пять ограничений, которые нужно назвать самому

- UNWIND_INFO v3 пока не поддерживается.
- Нет сопоставления metadata с машинным prolog/epilog.
- Нет большой независимой positive выборки и полной statistical accuracy.
- Dynamic mode видит только конкретный execution path.
- Проект не определяет malware и требует analyst review.

21. Ответы, которые нельзя придумывать

Если комиссия просит факт, которого нет в документации или проверках, лучший ответ — отделить известное от неизвестного и предложить процедуру измерения.

Нельзя говорить без данных	Корректный ответ
«Рынок составляет N миллионов»	Размер рынка не оценён; нужны сегментация и interviews.
«Точность 100%»	100% наблюдаемая specificity только на выбранных 106 benign files.
«Все protectors обнаруживаются»	Покрываются только patterns, попадающие в текущую model.
«V3 нигде не используется»	V3 опубликована Microsoft; распространённость нужно измерять corpus.
«LIEF разбирает хуже»	Нужно differential testing; априори превосходство не заявляется.
«Нейросеть даст хуже»	Без benchmark сравнение невозможно; rules лучше обоснованы для invariants.
«Проект первый в мире»	Найдена редкая комбинация функций; мировое первенство не доказано.
«MSVC точно пройдёт»	Нужен фактически выполненный независимый gate.
«Ошибок в коде нет»	Пройдены заявленные quality gates; абсолютная безошибочность невозможна.

Универсальный ответ при неизвестном факте

«Этого показателя в текущей работе нет, поэтому я не буду придумывать значение. Его можно получить следующим экспериментом: … В работе подтверждено только: …»

22. Итоговая позиция проекта

EH/Unwind Analyzer — самостоятельный специализированный инструмент структурного анализа Windows PE32+ AMD64 exception/unwind metadata. Он не конкурирует с IDA, Ghidra или WinDbg по широте; его преимущество — автоматический, объяснимый и воспроизводимый первый проход по узкой, технически важной области.

Что уже является сильным результатом

- Сквозная реализация от untrusted bytes до analyst report.
- 27 формализованных findings и 12 внутренних anomaly reasons.
- Разделение hard violations, heuristics, confidence и mitigations.
- Static cross-platform core и Windows runtime evidence.
- Schema-validated JSON, offline HTML, CSV и Ghidra integration.
- Развитая система tests, mutations, corpus и release artifacts.

Что необходимо для максимально сильного продукта

- Поддержать UNWIND_INFO v3.
- Провести differential testing и независимый MSVC gate.
- Добавить metadata-to-prolog consistency analysis.
- Собрать independently labeled corpus и измерить per-class accuracy.
- Провести market pilot и измерить сокращение analyst time.
- После этого развивать SARIF, plugins, backend/БД и frontend.

Финальный вывод
Проект достоин защиты не потому, что он «идеален» или «не имеет аналогов», а потому что сложная низкоуровневая задача доведена до законченной инженерной системы с объяснимыми решениями, воспроизводимыми результатами и честно установленными границами дальнейшего развития.